

Lab 1: Five Types of Constructor in C#

Title: Write a C# program to demonstrate five types of constructor in C#.

Theory

Constructors are special methods that initialize objects when they are created. They have the same name as the class and no return type.

Five types of constructors:

- **Default Constructor** - No parameters, initializes with default values
- **Parameterized Constructor** - Accepts parameters to set specific values
- **Copy Constructor** - Creates object by copying another object
- **Static Constructor** - Initializes static members, called once automatically
- **Private Constructor** - Prevents object creation from outside (used in Singleton pattern)

Code

```
1 using System;
2
3 class ConstructorDemo
4 {
5     private int value;
6     private string name;
7
8     // 1. Default Constructor
9     public ConstructorDemo()
10    {
11        value = 0;
12        name = "Default";
13        Console.WriteLine("Default Constructor:
14            value={0}, name={1}",
15                value, name);
16    }
17
18    // 2. Parameterized Constructor
19    public ConstructorDemo(int val, string nm)
20    {
21        value = val;
22        name = nm;
23        Console.WriteLine("Parameterized Constructor
24            : value={0}, name={1}",
25                value, name);
26    }
27
28    // 3. Copy Constructor
29    public ConstructorDemo(ConstructorDemo obj)
30    {
31        value = obj.value;
32        name = obj.name;
33        Console.WriteLine("Copy Constructor: value
34            ={0}, name={1}",
35                value, name);
36    }
37
38    // 4. Static Constructor
39    static ConstructorDemo()
40    {
41        Console.WriteLine("Static Constructor called
42            ");
43    }
```

```
39 }
40
41 // 5. Private Constructor
42 class Singleton
43 {
44     private static Singleton instance;
45
46     private Singleton()
47     {
48         Console.WriteLine("Private Constructor
49             called");
50     }
51
52     public static Singleton GetInstance()
53     {
54         if (instance == null)
55             instance = new Singleton();
56         return instance;
57     }
58 }
59
60 class Program
61 {
62     static void Main()
63     {
64         ConstructorDemo obj1 = new ConstructorDemo()
65             ;
66         ConstructorDemo obj2 = new ConstructorDemo
67             (10, "Test");
68         ConstructorDemo obj3 = new ConstructorDemo(
69             obj2);
70         Singleton s = Singleton.GetInstance();
71
72         Console.WriteLine("\nLab No.: 1");
73         Console.WriteLine("Name: Samir Paudel");
74         Console.WriteLine("Roll No./Section:
75             114-079/D");
76     }
77 }
```

Output

```
• [sam@arch Labs]$ dotnet Lab01.cs
Static Constructor called
Default Constructor: value=0, name=Default
Parameterized Constructor: value=10, name=Test
Copy Constructor: value=10, name=Test
Private Constructor called

Lab No.: 1
Name: Samir Paudel
Roll No./Section: 114-079/D
• [sam@arch Labs]$
```

Lab 2: Auto Property and Read-Only Property

Title

WAP in C# to demonstrate concept of auto Property and Read-Only Property.

Theory

Auto-implemented Properties: Provide a simplified syntax for property declaration where the compiler automatically creates a private backing field. Syntax: `public Type PropertyName { get; set; }`

Read-Only Properties: Properties that can only be read, not modified. They have only a `get` accessor. Read-only properties can be initialized in the constructor or with an initializer.

Auto properties make code more concise and eliminate the need to explicitly declare backing fields for simple properties.

Code

```
1 using System;
2
3 class Person
4 {
5     // Auto-implemented property with get and set
6     public string Name { get; set; }
7
8     // Auto-implemented property with default value
9     public int Age { get; set; } = 0;
10
11     // Read-only auto property (can only be set in
12     // constructor)
13     public string Country { get; }
14
15     // Read-only property with backing field
16     private string _ssn;
17     public string SSN
18     {
19         get { return _ssn; }
20     }
21
22     // Auto property with private setter
23     public string Email { get; private set; }
24
25     // Constructor
26     public Person(string country, string ssn)
27     {
28         Country = country; // Can set read-only
29                             // property in constructor
30         _ssn = ssn;
31     }
32
33     public void SetEmail(string email)
34     {
35         Email = email; // Can set via method since
36                         // setter is private
37     }
38 }
```

```
36 public void DisplayInfo()
37 {
38     Console.WriteLine($"Name: {Name}");
39     Console.WriteLine($"Age: {Age}");
40     Console.WriteLine($"Country: {Country}");
41     Console.WriteLine($"SSN: {SSN}");
42     Console.WriteLine($"Email: {Email}");
43 }
44
45 class Program
46 {
47     static void Main()
48     {
49         Person person = new Person("Nepal", "
50             123-45-6789");
51
52         // Setting auto properties
53         person.Name = "Ram Sharma";
54         person.Age = 25;
55         person.SetEmail("ram@example.com");
56
57         Console.WriteLine("Person Information:");
58         person.DisplayInfo();
59
60         // Attempting to modify read-only property
61         // will cause compile error
62         // person.Country = "India"; // Error!
63         // person.SSN = "987-65-4321"; // Error!
64
65         Console.WriteLine("\nLab No.: 2");
66         Console.WriteLine("Name: Samir Paudel");
67         Console.WriteLine("Roll No./Section:
68             114-079/D");
69     }
70 }
```

Output

```
• [sam@arch Labs]$ dotnet Lab02.cs
Person Information:
Name: Ram Sharma
Age: 25
Country: Nepal
SSN: 123-45-6789
Email: ram@example.com

Lab No.: 2
Name: Samir Paudel
Roll No./Section: 114-079/D
[am@arch Labs]$
```

Lab 3: Jagged Array

Title

WAP in C# to demonstrate jagged array.

Theory

A **jagged array** is an array of arrays where each element is itself an array. Unlike multidimensional arrays, the arrays in a jagged array can have different lengths.

Syntax: `type[][] arrayName = new type[rows][];`

Jagged arrays are useful when you need to store rows of data with varying numbers of columns, making them more memory-efficient than rectangular arrays for irregular data structures.

Code

```
1 using System;
2
3 class JaggedArrayDemo
4 {
5     static void Main()
6     {
7         // Declare jagged array with 4 rows
8         int[][] jaggedArray = new int[4][];
9
10        // Initialize each row with different
11        // lengths
12        jaggedArray[0] = new int[3] { 1, 2, 3 };
13        jaggedArray[1] = new int[5] { 4, 5, 6, 7, 8 };
14        jaggedArray[2] = new int[2] { 9, 10 };
15        jaggedArray[3] = new int[4] { 11, 12, 13, 14 };
16
17        Console.WriteLine("Jagged Array Elements:");
18        Console.WriteLine("=====");
19
20        // Display jagged array
21        for (int i = 0; i < jaggedArray.Length; i++)
22        {
23            Console.Write($"Row {i}: ");
24            for (int j = 0; j < jaggedArray[i].
25                Length; j++)
26            {
27                Console.Write(jaggedArray[i][j] + "
28                    ");
29            }
30            Console.WriteLine();
31        }
32
33        // Another example with user input
34        // simulation
35        Console.WriteLine("\nStudent Marks (
36            Different subjects per student):");
37
38        Console.WriteLine("
39            =====");
40
41        int[][] studentMarks = new int[3][];
42        studentMarks[0] = new int[] { 85, 90, 78, 92
43            }; // 4 subjects
44        studentMarks[1] = new int[] { 88, 76 };
45            // 2 subjects
46        studentMarks[2] = new int[] { 95, 89, 91 };
47            // 3 subjects
48
49        for (int i = 0; i < studentMarks.Length; i
50            ++){
51            Console.WriteLine($"Student {i + 1} marks: "
52                );
53            double sum = 0;
54            for (int j = 0; j < studentMarks[i].
55                Length; j++){
56                Console.WriteLine(studentMarks[i][j] + "
57                    ");
58                sum += studentMarks[i][j];
59            }
60            double average = sum / studentMarks[i].
61                Length;
62            Console.WriteLine($"(Average: {average:
63                F2})");
64        }
65
66        Console.WriteLine("\nLab No.: 3");
67        Console.WriteLine("Name: Samir Paudel");
68        Console.WriteLine("Roll No./Section:
69            114-079/D");
70    }
71}
```

Output

```
• [sam@arch Labs]$ dotnet Lab03.cs
Jagged Array Elements:
=====
Row 0: 1 2 3
Row 1: 4 5 6 7 8
Row 2: 9 10
Row 3: 11 12 13 14

Student Marks:
Student 1: 85 90 78 92 (Avg: 86.25)
Student 2: 88 76 (Avg: 82.00)
Student 3: 95 89 91 (Avg: 91.67)

Lab No.: 3
Name: Samir Paudel
Roll No./Section: 114-079/D
❖ [sam@arch Labs]$
```

Lab 4: Indexer in C#

Title

WAP to demonstrate Indexer in C#:

- When index is of int type
- When index is of other than int type.

Theory

Indexers allow objects to be indexed like arrays. They enable instances of a class to be accessed using array-like syntax.

Syntax: `type this[type index] { get {} set {} }`

Indexers can be of different types:

- **Integer indexer** - Uses int type for index (most common)
- **String indexer** - Uses string type for index (like dictionary)
- **Other types** - Can use any type as index parameter

Code

```
1 using System;
2
3 // Example 1: Integer Indexer
4 class StudentMarks
5 {
6     private int[] marks = new int[5];
7
8     // Integer indexer
9     public int this[int index]
10    {
11        get
12        {
13            if (index >= 0 && index < marks.Length)
14                return marks[index];
15            else
16                throw new IndexOutOfRangeException("Index out of range");
17        }
18        set
19        {
20            if (index >= 0 && index < marks.Length)
21                marks[index] = value;
22            else
23                throw new IndexOutOfRangeException("Index out of range");
24        }
25    }
26
27    public int Length
28    {
29        get { return marks.Length; }
30    }
31 }
32
33 // Example 2: String Indexer
34 class PhoneBook
35 {
36     private string[] names = new string[10];
37     private string[] phones = new string[10];
38     private int count = 0;
39
40     // String indexer
41     public string this[string name]
42     {
43         get
44         {
45             for (int i = 0; i < count; i++)
46             {
47                 if (names[i] == name)
48                     return phones[i];
49             }
50             return "Not Found";
51         }
52         set
53         {
```

```
54         for (int i = 0; i < count; i++)
55         {
56             if (names[i] == name)
57             {
58                 phones[i] = value;
59                 return;
60             }
61         }
62         // Add new entry
63         if (count < names.Length)
64         {
65             names[count] = name;
66             phones[count] = value;
67             count++;
68         }
69     }
70 }
71
72 public void Display()
73 {
74     Console.WriteLine("\nPhone Book Entries:");
75     for (int i = 0; i < count; i++)
76     {
77         Console.WriteLine($"{names[i]}: {phones[i]}");
78     }
79 }
80
81 class Program
82 {
83     static void Main()
84     {
85         // Integer Indexer Demo
86         Console.WriteLine("=== Integer Indexer Demo ===");
87         StudentMarks student = new StudentMarks();
88
89         student[0] = 85;
90         student[1] = 90;
91         student[2] = 78;
92         student[3] = 92;
93         student[4] = 88;
94
95         Console.WriteLine("Student Marks:");
96         for (int i = 0; i < student.Length; i++)
97         {
98             Console.WriteLine($"Subject {i + 1}: {student[i]}");
99         }
100
101         // String Indexer Demo
102         Console.WriteLine("\n=== String Indexer Demo ===");
103         PhoneBook phoneBook = new PhoneBook();
```

<pre> 105 phoneBook["Ram"] = "9841234567"; 106 phoneBook["Sita"] = "9847654321"; 107 phoneBook["Hari"] = "9851112222"; 108 phoneBook["Gita"] = "9863334444"; 109 110 phoneBook.Display(); 111 112 Console.WriteLine(\$"\\nRam's phone: { 113 phoneBook["Ram"]}"); 114 Console.WriteLine(\$"\\nSita's phone: {phoneBook </pre>	<pre> 115 ["Sita"]}"); 116 Console.WriteLine(\$"\\nUnknown: {phoneBook[" 117 Unknown"]}"); 118 119 Console.WriteLine(\$"\\nLab No.: 4"); 120 Console.WriteLine(\$"\\nName: Samir Paudel"); 121 Console.WriteLine(\$"\\nRoll No./Section: 122 114-079/D"); 123 } </pre>
---	--

Output

• [sam@arch Labs]\$ dotnet Lab04.cs

=== Integer Indexer ===

Subject 1: 85

Subject 2: 90

Subject 3: 78

Subject 4: 0

Subject 5: 0

=== String Indexer ===

Phone Book:

Ram: 9841234567

Sita: 9847654321

Ram's phone: 9841234567

Lab No.: 4

Name: Samir Paudel

Roll No./Section: 114-079/D

1

Lab 5: Base Keyword in C#

Title

WAP to demonstrate:

- The use of base keyword to access base class fields
- The use of base keyword to call base class methods
- The use of base keyword to call base class constructor

Theory

The **base** keyword is used to access members of the base class from within a derived class.

Three main uses of base keyword:

- **Access base class fields** - When derived class has a field with the same name
- **Call base class methods** - To invoke an overridden method from the base class
- **Call base class constructor** - To invoke a specific base class constructor

The base keyword is essential for extending base class functionality while maintaining access to original implementations.

Code

```
1 using System;
2
3 // Base class
4 class Vehicle
5 {
6     protected string brand = "Generic Brand";
7     protected int speed = 0;
8
9     public Vehicle()
10    {
11        Console.WriteLine("Vehicle: Default
12                           constructor called");
13    }
14
15    public Vehicle(string brand, int speed)
16    {
17        this.brand = brand;
18        this.speed = speed;
19        Console.WriteLine($"Vehicle: Parameterized
20                           constructor called");
21    }
22
23    public virtual void ShowInfo()
24    {
25        Console.WriteLine($"Brand: {brand}, Speed: {
26                           speed} km/h");
27    }
28
29    public void StartEngine()
30    {
31        Console.WriteLine($"{{brand}}: Engine started"
32                           );
33    }
34 }
35
36 // Derived class
37 class Car : Vehicle
38 {
39     private string brand = "Car Brand"; // Hides
40     // base class field
41     private int numberOfDoors;
42
43     // a) Using base keyword to call base class
44     // constructor
45     public Car() : base()
46     {
47         numberOfDoors = 4;
48         Console.WriteLine("Car: Default constructor
49                           called");
50     }
51
52     public Car(string brand, int speed, int doors) :
53
54         base(brand, speed)
55     {
56         numberOfDoors = doors;
57         Console.WriteLine("Car: Parameterized
58                           constructor called");
59     }
60
61     // b) Using base keyword to call base class
62     // method
63     public override void ShowInfo()
64     {
65         Console.WriteLine("--- Car Information ---");
66         ;
67         base.ShowInfo(); // Call base class
68         ShowInfo()
69         Console.WriteLine($"Number of Doors: {
70                           numberOfDoors}");
71     }
72
73     // c) Using base keyword to access base class
74     // field
75     public void DisplayBrands()
76     {
77         Console.WriteLine($"Base class brand: {base.
78                           brand}");
79         Console.WriteLine($"Derived class brand: {
80                           this.brand}");
81     }
82
83     public void StartCarEngine()
84     {
85         base.StartEngine(); // Call base class
86         // method
87         Console.WriteLine("Car-specific systems
88                           initialized");
89     }
90 }
91
92 class Program
93 {
94     static void Main()
95     {
96         Console.WriteLine("=== Creating Car with
97                           default constructor ===");
98         Car car1 = new Car();
99         Console.WriteLine();
100
101         Console.WriteLine("=== Creating Car with
102                           parameterized constructor ===");
103         Car car2 = new Car("Toyota", 180, 4);
104         Console.WriteLine();
105
106         Console.WriteLine("=== Demonstrating base
```

```

86         keyword uses ==="\n");
87     // a) Accessing base class fields using base
      keyword
88     Console.WriteLine("1. Accessing base class
      fields:");
89     car2.DisplayBrands();
90     Console.WriteLine();
91
92     // b) Calling base class method using base
      keyword
93     Console.WriteLine("2. Calling base class
      method:");
94     car2.ShowInfo();
95     Console.WriteLine();

```

```

96
97     // c) Using base keyword in constructor (
      already demonstrated above)
98     Console.WriteLine("3. Starting engine (calls
      base class method:");
99     car2.StartCarEngine();
100
101     Console.WriteLine("\nLab No.: 5");
102     Console.WriteLine("Name: Samir Paudel");
103     Console.WriteLine("Roll No./Section:
      114-079/D");
104 }
105 }

```

Output

=== Creating Car ===

Vehicle: Parameterized constructor

Car: Parameterized constructor

=== Base Keyword Uses ===

1. Accessing fields:

Base brand: Toyota

Derived brand: Car Brand

2. Calling method:

--- Car Information ---

Brand: Toyota, Speed: 180 km/h

Doors: 4

3. Starting engine:

Toyota: Engine started

Car systems initialized

Lab No.: 5

Name: Samir Paudel

Roll No./Section: 114-079/D

❖[sam@arch Labs]\$ █

1

Lab 6: Method Overriding and Dynamic Polymorphism

Title

Program to show:

- method overriding and method hiding/shadowing in C#
- dynamic polymorphism using method overriding.

Theory

Method Overriding: Allows a derived class to provide a specific implementation of a method that is already defined in its base class. Uses `virtual` and `override` keywords.

Method Hiding/Shadowing: Uses the `new` keyword to hide a base class method with the same name in a derived class.

Dynamic Polymorphism: Also known as runtime polymorphism, achieved through method overriding. The method to be called is determined at runtime based on the actual object type.

Key differences:

- Override maintains polymorphic behavior
- New breaks polymorphism and hides the base method

Code

```
1 using System;
2
3 // Base class
4 class Shape
5 {
6     protected string name;
7
8     public Shape(string name)
9     {
10         this.name = name;
11     }
12
13     // Virtual method for overriding
14     public virtual void Draw()
15     {
16         Console.WriteLine($"Drawing a {name}");
17     }
18
19     // Method for hiding demonstration
20     public void ShowInfo()
21     {
22         Console.WriteLine($"This is a {name}");
23     }
24
25     public virtual double CalculateArea()
26     {
27         return 0;
28     }
29 }
30
31 // Derived class 1: Circle
32 class Circle : Shape
33 {
34     private double radius;
35
36     public Circle(double radius) : base("Circle")
37     {
38         this.radius = radius;
39     }
40
41     // Method Overriding using override keyword
42     public override void Draw()
43     {
44         Console.WriteLine($"Drawing a circle with
45             radius {radius}");
46     }
47
48     public override double CalculateArea()
49     {
50         return Math.PI * radius * radius;
51     }
52 }
```

```
52 // Method Hiding using new keyword
53 public new void ShowInfo()
54 {
55     Console.WriteLine($"This is a Circle with
56         radius {radius}");
57 }
58
59 // Derived class 2: Rectangle
60 class Rectangle : Shape
61 {
62     private double length, width;
63
64     public Rectangle(double length, double width) :
65         base("Rectangle")
66     {
67         this.length = length;
68         this.width = width;
69     }
70
71     // Method Overriding
72     public override void Draw()
73     {
74         Console.WriteLine($"Drawing rectangle: {
75             length} x {width}");
76     }
77
78     public override double CalculateArea()
79     {
80         return length * width;
81     }
82
83     // Method Hiding
84     public new void ShowInfo()
85     {
86         Console.WriteLine($"This is a Rectangle: {
87             length} x {width}");
88     }
89 }
90
91 class Program
92 {
93     static void Main()
94     {
95         Console.WriteLine("=== Method Overriding vs
96             Method Hiding ===\n");
97
98         Circle circle = new Circle(5.0);
99         Rectangle rectangle = new Rectangle(4.0,
100             6.0);
101
102         // Direct calls
103         Console.WriteLine("--- Direct Method Calls
```



```

99         ---");
100         circle.Draw();
101         circle.ShowInfo();
102         Console.WriteLine($"Circle Area: {circle.
103             CalculateArea():F2}\n");
104
105         rectangle.Draw();
106         rectangle.ShowInfo();
107         Console.WriteLine($"Rectangle Area: {
108             rectangle.CalculateArea():F2}\n");
109
110         // Dynamic Polymorphism (Runtime
111         Polymorphism)
112         Console.WriteLine("--- Dynamic Polymorphism
113             ---");
114         Shape shape1 = new Circle(7.0);
115         Shape shape2 = new Rectangle(5.0, 8.0);
116
117         // Overridden methods show polymorphic
118         behavior
119         Console.WriteLine("Using base class
120             reference:");
121         shape1.Draw(); // Calls Circle's Draw()
122         Console.WriteLine($"Area: {shape1.
123             CalculateArea():F2}");
124
125         shape2.Draw(); // Calls Rectangle's Draw()
126         Console.WriteLine($"Area: {shape2.
127             CalculateArea():F2}");
128
129         Console.WriteLine();
130
131         // Hidden methods do NOT show polymorphic

```

```

123         behavior
124         Console.WriteLine("Hidden method (new
125             keyword) behavior:");
126         shape1.ShowInfo(); // Calls Shape's
127             ShowInfo(), not Circle's
128         shape2.ShowInfo(); // Calls Shape's
129             ShowInfo(), not Rectangle's
130
131         Console.WriteLine();
132
133         // Array of shapes demonstrating dynamic
134         polymorphism
135         Console.WriteLine("--- Array of Shapes (
136             Dynamic Polymorphism) ---");
137         Shape[] shapes = new Shape[3];
138         shapes[0] = new Circle(3.0);
139         shapes[1] = new Rectangle(4.0, 5.0);
140         shapes[2] = new Circle(6.0);
141
142         foreach (Shape s in shapes)
143         {
144             s.Draw();
145             Console.WriteLine($"Area: {s.
146                 CalculateArea():F2}\n");
147         }
148
149         Console.WriteLine("Lab No.: 6");
150         Console.WriteLine("Name: Samir Paudel");
151         Console.WriteLine("Roll No./Section:
152             114-079/D");
153     }
154 }

```

Output

```

[sam@arch Labs]$ dotnet Lab06.cs
=== Method Overriding vs Hiding ===

```

```

Drawing circle with radius 5
Circle with radius 5
Area: 78.54

```

```

=== Dynamic Polymorphism ===
Drawing circle with radius 7
Area: 153.94
Drawing rectangle: 5 x 8
Area: 40.00

```

```

=== Hidden Method Behavior ===
This is a Circle
This is a Rectangle

```

```

Lab No.: 6
Name: Samir Paudel
Roll No./Section: 114-079/D

```

```

[sam@arch Labs]$

```

Lab 7: Abstract Class, Interface, and Multiple Inheritance

Title

WAP to illustrate the concept of:

- Abstract class
- Interface
- Multiple inheritance using interface in C#

Theory

Abstract Class: A class that cannot be instantiated and may contain both abstract and non-abstract methods. Used as a base class. Declared using **abstract** keyword.

Interface: A contract that defines a set of methods and properties without implementation. Classes implementing an interface must provide implementation for all its members.

Multiple Inheritance: C# doesn't support multiple inheritance with classes but supports it through interfaces. A class can implement multiple interfaces.

Key differences:

- Abstract class can have implementation; interface cannot (before C# 8.0)
- A class can inherit only one abstract class but multiple interfaces
- Abstract class can have fields; interface cannot

Code

```
1 using System;
2
3 // a) Abstract Class
4 abstract class Animal
5 {
6     protected string name;
7
8     public Animal(string name)
9     {
10         this.name = name;
11     }
12
13     // Abstract method (must be implemented by
14     // derived classes)
15     public abstract void MakeSound();
16
17     // Non-abstract method (can be used by derived
18     // classes)
19     public void Sleep()
20     {
21         Console.WriteLine($"{name} is sleeping...");
22     }
23
24     // Abstract method
25     public abstract void Move();
26 }
27
28 // b) Interfaces
29 interface IFlyable
30 {
31     void Fly();
32     int MaxAltitude { get; set; }
33 }
34
35 interface ISwimmable
36 {
37     void Swim();
38     int MaxDepth { get; set; }
39 }
40
41 interface IRunnable
42 {
43     void Run();
44 }
45
46 // Derived class from abstract class
47 class Dog : Animal, IRunnable
48 {
49 }
```

```
47 public Dog(string name) : base(name) { }
48
49 public override void MakeSound()
50 {
51     Console.WriteLine($"{name} says: Woof! Woof!");
52 }
53
54 public override void Move()
55 {
56     Console.WriteLine($"{name} is running on
57         four legs");
58 }
59
60 public void Run()
61 {
62     Console.WriteLine($"{name} is running fast!");
63 }
64
65 // c) Multiple Inheritance using Interfaces
66 class Duck : Animal, IFlyable, ISwimmable
67 {
68     public int MaxAltitude { get; set; }
69     public int MaxDepth { get; set; }
70
71     public Duck(string name) : base(name)
72     {
73         MaxAltitude = 1000;
74         MaxDepth = 10;
75     }
76
77     public override void MakeSound()
78     {
79         Console.WriteLine($"{name} says: Quack!
80             Quack!");
81     }
82
83     public override void Move()
84     {
85         Console.WriteLine($"{name} can walk, fly,
86             and swim");
87     }
88
89     public void Fly()
90     {
91         Console.WriteLine($"{name} is flying up to {
92             MaxAltitude} meters");
93     }
94 }
```

```

90     }
91
92     public void Swim()
93     {
94         Console.WriteLine($"{name} is swimming down
95             to {MaxDepth} meters");
96     }
97 }
98 class Bird : Animal, IFlyable
99 {
100     public int MaxAltitude { get; set; }
101
102     public Bird(string name) : base(name)
103     {
104         MaxAltitude = 5000;
105     }
106
107     public override void MakeSound()
108     {
109         Console.WriteLine($"{name} says: Tweet!
110             Tweet!");
111     }
112
113     public override void Move()
114     {
115         Console.WriteLine($"{name} is flying");
116     }
117
118     public void Fly()
119     {
120         Console.WriteLine($"{name} is flying at {
121             MaxAltitude} meters altitude");
122     }
123 }
124 class Program
125 {
126     static void Main()
127     {
128         Console.WriteLine("=== Abstract Class Demo
129             ===\n");
130
131         Dog dog = new Dog("Buddy");
132         dog.MakeSound();
133         dog.Move();
134         dog.Sleep();
135         dog.Run();

```

```

134
135         Console.WriteLine("\n=== Interface Demo ===\n");
136
137         Bird bird = new Bird("Sparrow");
138         bird.MakeSound();
139         bird.Move();
140         bird.Fly();
141         bird.Sleep();
142
143         Console.WriteLine("\n=== Multiple
144             Inheritance using Interface ===\n");
145
146         Duck duck = new Duck("Donald");
147         duck.MakeSound();
148         duck.Move();
149         duck.Fly();
150         duck.Swim();
151         duck.Sleep();
152
153         Console.WriteLine("\n=== Polymorphism with
154             Abstract Class ===\n");
155
156         Animal[] animals = { dog, bird, duck };
157         foreach (Animal animal in animals)
158         {
159             animal.MakeSound();
160             animal.Move();
161             Console.WriteLine();
162         }
163
164         Console.WriteLine("=== Interface Reference
165             ===\n");
166
167         IFlyable[] flyingThings = { bird, duck };
168         foreach (IFlyable flyer in flyingThings)
169         {
170             flyer.Fly();
171         }
172
173         Console.WriteLine("\nLab No.: 7");
174         Console.WriteLine("Name: Samir Paudel");
175         Console.WriteLine("Roll No./Section:
176             114-079/D");
177     }
178 }

```

Output

```

[sam@arch Labs]$ dotnet Lab07.cs
=== Abstract Class Demo ===
Buddy says: Woof!
Buddy is running
Buddy runs fast!

=== Multiple Inheritance Demo ===
Donald says: Quack!
Donald can walk, fly, and swim
Donald flies to 1000m
Donald swims to 10m depth

=== Polymorphism Demo ===
Buddy says: Woof!
Buddy is running
Donald says: Quack!
Donald can walk, fly, and swim

Lab No.: 7
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 8: Structure, Enumeration, and Partial Class

Title

WAP program that contains:

- Structure (struct)
- Enumeration (enum)
- Partial class

Theory

Structure (struct): A value type that can contain data members and function members. Structures are stored on the stack and are suitable for small data structures.

Enumeration (enum): A value type consisting of a set of named constants. Used to define a group of related integral constants.

Partial Class: Allows splitting the definition of a class across multiple files. All parts must use the **partial** keyword and will be combined during compilation.

Key points:

- Struct is value type; class is reference type
- Enum provides readable names for numeric values
- Partial classes help organize large classes and support code generation

Code

```
1 using System;
2
3 // a) Structure (struct)
4 struct Point
5 {
6     public int X;
7     public int Y;
8
9     public Point(int x, int y)
10    {
11        X = x;
12        Y = y;
13    }
14
15    public void Display()
16    {
17        Console.WriteLine($"Point: ({X}, {Y})");
18    }
19
20    public double DistanceFromOrigin()
21    {
22        return Math.Sqrt(X * X + Y * Y);
23    }
24 }
25
26 struct Student
27 {
28     public int RollNo;
29     public string Name;
30     public double Marks;
31
32     public Student(int rollNo, string name, double marks)
33     {
34         RollNo = rollNo;
35         Name = name;
36         Marks = marks;
37     }
38
39     public void DisplayInfo()
40     {
41         Console.WriteLine($"Roll No: {RollNo}");
42         Console.WriteLine($"Name: {Name}");
43         Console.WriteLine($"Marks: {Marks}");
44         Console.WriteLine($"Grade: {GetGrade()}");
45     }
46
47     public string GetGrade()
```

```
48     {
49         if (Marks >= 90) return "A+";
50         else if (Marks >= 80) return "A";
51         else if (Marks >= 70) return "B+";
52         else if (Marks >= 60) return "B";
53         else if (Marks >= 50) return "C";
54         else return "F";
55     }
56 }
57
58 // b) Enumeration (enum)
59 enum Days
60 {
61     Sunday,
62     Monday,
63     Tuesday,
64     Wednesday,
65     Thursday,
66     Friday,
67     Saturday
68 }
69
70 enum OrderStatus
71 {
72     Pending = 1,
73     Processing = 2,
74     Shipped = 3,
75     Delivered = 4,
76     Cancelled = 5
77 }
78
79 enum Priority
80 {
81     Low = 1,
82     Medium = 2,
83     High = 3,
84     Critical = 4
85 }
86
87 // c) Partial Class (Part 1)
88 partial class Employee
89 {
90     private int empId;
91     private string name;
92     private double salary;
93
94     public Employee(int id, string name, double salary)
95     {
```

```

96         this.empId = id;
97         this.name = name;
98         this.salary = salary;
99     }
100
101     public void DisplayBasicInfo()
102     {
103         Console.WriteLine($"Employee ID: {empId}");
104         Console.WriteLine($"Name: {name}");
105         Console.WriteLine($"Salary: {salary:C}");
106     }
107 }
108
109 // c) Partial Class (Part 2)
110 partial class Employee
111 {
112     public void CalculateBonus()
113     {
114         double bonus = salary * 0.10;
115         Console.WriteLine($"Bonus: {bonus:C}");
116         Console.WriteLine($"Total: {(salary + bonus):C}");
117     }
118
119     public void GiveRaise(double percentage)
120     {
121         double raise = salary * (percentage / 100);
122         salary += raise;
123         Console.WriteLine($"Salary increased by {percentage}%");
124         Console.WriteLine($"New Salary: {salary:C}");
125     }
126 }
127
128 class Program
129 {
130     static void Main()
131     {
132         Console.WriteLine("=== STRUCTURE Demo ===\n");
133
134         Point p1 = new Point(3, 4);
135         p1.Display();
136         Console.WriteLine($"Distance from origin: {p1.DistanceFromOrigin():F2}");
137
138         Console.WriteLine();
139
140         Student student = new Student(101, "Ram Kumar", 85.5);
141         student.DisplayInfo();
142
143         Console.WriteLine("\n=== ENUMERATION Demo ===\n");
144
145         // Days enum
146         Days today = Days.Wednesday;
147         Console.WriteLine($"Today is: {today}");
148         Console.WriteLine($"Integer value: {(int)today}");
149
150         Console.WriteLine("\nAll days of the week:");

```

```

151         foreach (Days day in Enum.GetValues(typeof(Days)))
152         {
153             Console.WriteLine($"{(int)day}: {day}");
154         }
155
156         Console.WriteLine();
157
158         // OrderStatus enum
159         OrderStatus status = OrderStatus.Processing;
160         Console.WriteLine($"Order Status: {status} (Code: {(int)status})");
161
162         switch (status)
163         {
164             case OrderStatus.Pending:
165                 Console.WriteLine("Order is pending approval");
166                 break;
167             case OrderStatus.Processing:
168                 Console.WriteLine("Order is being processed");
169                 break;
170             case OrderStatus.Shipped:
171                 Console.WriteLine("Order has been shipped");
172                 break;
173             case OrderStatus.Delivered:
174                 Console.WriteLine("Order delivered successfully");
175                 break;
176             default:
177                 Console.WriteLine("Order status unknown");
178                 break;
179         }
180
181         Console.WriteLine();
182
183         // Priority enum
184         Priority taskPriority = Priority.High;
185         Console.WriteLine($"Task Priority: {taskPriority} (Level: {(int)taskPriority})");
186
187         Console.WriteLine("\n=== PARTIAL CLASS Demo ===\n");
188
189         Employee emp = new Employee(1001, "Shyam Sharma", 50000);
190         emp.DisplayBasicInfo();
191         Console.WriteLine();
192         emp.CalculateBonus();
193         Console.WriteLine();
194         emp.GiveRaise(15);
195
196         Console.WriteLine("\nLab No.: 8");
197         Console.WriteLine("Name: Samir Paudel");
198         Console.WriteLine("Roll No./Section: 114-079/D");
199     }
200 }

```

Output

```

[sam@arch Labs]$ dotnet Lab08.cs
=== STRUCTURE ===
Point: (3, 4)
Distance: 5.00

=== ENUMERATION ===
Today: Wednesday (3)
Status: Processing (Code: 2)
Priority: High (Level: 3)

=== PARTIAL CLASS ===
ID: 101, Name: Ram Sharma, Salary: $50,000.00
Bonus: $5,000.00, Total: $55,000.00
New Salary: $57,500.00

Lab No.: 8
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 9: Delegates, Multicast Delegates, and Events

Title

WAP to illustrate the concept of:

- a) Delegate
- b) Multicast delegate
- c) Func Delegate
- d) Action Delegate
- e) Anonymous Method
- f) Event in C#

Theory

Delegate: A type-safe function pointer that can reference methods with a specific signature.

Multicast Delegate: A delegate that can reference multiple methods. All referenced methods are called when the delegate is invoked.

Func Delegate: A built-in generic delegate that returns a value. Last type parameter is the return type.

Action Delegate: A built-in generic delegate that doesn't return a value (void).

Anonymous Method: A method without a name, defined using the `delegate` keyword.

Event: A mechanism for communication between objects based on the publisher-subscriber model. Events use delegates.

Code

```
1 using System;
2
3 // a) Delegate declaration
4 delegate int MathOperation(int a, int b);
5 delegate void DisplayMessage(string message);
6
7 // Event publisher class
8 class Button
9 {
10     // f) Event declaration
11     public event EventHandler Click;
12
13     public void OnClick()
14     {
15         Console.WriteLine("Button clicked!");
16         // Raise the event
17         if (Click != null)
18         {
19             Click(this, EventArgs.Empty);
20         }
21     }
22 }
23
24 class Calculator
25 {
26     // Methods for delegates
27     public int Add(int a, int b)
28     {
29         return a + b;
30     }
31
32     public int Subtract(int a, int b)
33     {
34         return a - b;
35     }
36
37     public int Multiply(int a, int b)
38     {
39         return a * b;
40     }
41
42     public static void ShowResult(string message)
43     {
44         Console.WriteLine($"Result: {message}");
45     }
46 }
47
48 class Program
49 {
```

```
50     static void Main()
51     {
52         Calculator calc = new Calculator();
53
54         // a) Delegate Demo
55         Console.WriteLine("=== DELEGATE Demo ===\n");
56
57         MathOperation operation = calc.Add;
58         int result = operation(10, 5);
59         Console.WriteLine($"10 + 5 = {result}");
60
61         operation = calc.Subtract;
62         result = operation(10, 5);
63         Console.WriteLine($"10 - 5 = {result}");
64
65         operation = calc.Multiply;
66         result = operation(10, 5);
67         Console.WriteLine($"10 * 5 = {result}");
68
69         // b) Multicast Delegate Demo
70         Console.WriteLine("\n=== MULTICAST DELEGATE Demo ===\n");
71
72         DisplayMessage display = Calculator.ShowResult;
73         display += msg => Console.WriteLine($"Display: {msg}");
74         display += msg => Console.WriteLine($"Log: {msg}");
75
76         Console.WriteLine("Calling multicast delegate:");
77         display("Hello from multicast delegate");
78
79         // c) Func Delegate Demo
80         Console.WriteLine("\n=== FUNC DELEGATE Demo ===\n");
81
82         // Func with two int parameters and int return
83         Func<int, int, int> add = (x, y) => x + y;
84         Func<int, int, int> multiply = (x, y) => x * y;
85
86         Console.WriteLine($"Func Add: 15 + 10 = {add(15, 10)}");
87         Console.WriteLine($"Func Multiply: 15 * 10 = {multiply(15, 10)}");
```

```

88 // Func with string return
89 Func<string, string> greet = name => $"Hello
90     , {name}!";
91 Console.WriteLine(greet("Ram"));
92
93 // d) Action Delegate Demo
94 Console.WriteLine("\n=== ACTION DELEGATE
95     Demo ===\n");
96
97 // Action with no return value
98 Action<string> printMessage = message =>
99     Console.WriteLine($"Action: {message}");
100 printMessage("This is an Action delegate");
101
102 Action<int, int> printSum = (a, b) =>
103     Console.WriteLine($"Sum of {a} and {b}
104     is {a + b}");
105 printSum(20, 30);
106
107 Action sayHello = () => Console.WriteLine("
108     Hello from Action!");
109 sayHello();
110
111 // e) Anonymous Method Demo
112 Console.WriteLine("\n=== ANONYMOUS METHOD
113     Demo ===\n");
114
115 MathOperation subtract = delegate(int x, int
116     y)
117 {
118     Console.WriteLine($"Anonymous method: {x
119         } - {y}");
120     return x - y;
121 };
122
123 result = subtract(50, 20);
124 Console.WriteLine($"Result: {result}");
125
126 DisplayMessage anonymousDisplay = delegate(
127     string msg)
128 {
129     Console.WriteLine($"Anonymous: {msg}");
130     Console.WriteLine("This is an anonymous
131         method!");
132 };
133 anonymousDisplay("Testing anonymous method")
134     ;
135
136 // f) Event Demo
137 Console.WriteLine("\n=== EVENT Demo ===\n");
138
139 Button button = new Button();
140
141 // Subscribe to event

```

```

133 button.Click += Button_Click;
134 button.Click += (sender, e) =>
135     Console.WriteLine("Event: Lambda handler
136         executed");
137
138 button.Click += delegate(object sender,
139     EventArgs e)
140 {
141     Console.WriteLine("Event: Anonymous
142         method handler executed");
143 };
144
145 // Trigger event
146 button.OnClick();
147
148 Console.WriteLine("\n=== Practical Example:
149     Calculator with Delegates ===\n");
150
151 Func<int, int, int> calculate;
152 string operation_choice = "add";
153
154 switch (operation_choice)
155 {
156     case "add":
157         calculate = (a, b) => a + b;
158         break;
159     case "subtract":
160         calculate = (a, b) => a - b;
161         break;
162     case "multiply":
163         calculate = (a, b) => a * b;
164         break;
165     default:
166         calculate = (a, b) => 0;
167         break;
168 }
169
170 Console.WriteLine($"Result: {calculate(100,
171     50)}");
172
173 Console.WriteLine("\nLab No.: 9");
174 Console.WriteLine("Name: Samir Paudel");
175 Console.WriteLine("Roll No./Section:
176     114-079/D");
177
178 // Event handler method
179 static void Button_Click(object sender,
180     EventArgs e)
181 {
182     Console.WriteLine("Event: Button_Click
183         handler executed");
184 }

```

Output

```

=== DELEGATE ===
10 + 5 = 15

=== MULTICAST DELEGATE ===
Result: Hello multicast
Log: Hello multicast

=== FUNC DELEGATE ===
15 + 10 = 25
Hello, Ram!

=== ACTION DELEGATE ===
Action: This is Action delegate
20 + 30 = 50

=== ANONYMOUS METHOD ===
Anonymous: 50 - 20
Result: 30

=== EVENT ===
Button clicked!
Button_Click handler
Lambda handler

Lab No.: 9
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 10: Generic and Non-Generic Collections

Title

WAP which use any:

- Non generic collection
- Generic Collection

Theory

Non-Generic Collections: Collections that store elements as object type. Found in `System.Collections` namespace. Examples: `ArrayList`, `Hashtable`, `Stack`, `Queue`.

Generic Collections: Type-safe collections that use generic type parameters. Found in `System.Collections.Generic` namespace. Examples: `List<T>`, `Dictionary<TKey, TValue>`, `Stack<T>`, `Queue<T>`.

Advantages of Generic Collections:

- Type safety - No need for casting
- Better performance - No boxing/unboxing for value types
- Code reusability
- Compile-time type checking

Code

```
1 using System;
2 using System.Collections;           // Non-generic
   collections
3 using System.Collections.Generic;   // Generic
   collections
4
5 class Program
6 {
7     static void Main()
8     {
9         // a) NON-GENERIC COLLECTIONS
10        Console.WriteLine("=== NON-GENERIC
           COLLECTIONS ===\n");
11
12        // ArrayList Demo
13        Console.WriteLine("--- ArrayList Demo ---");
14        ArrayList arrayList = new ArrayList();
15
16        arrayList.Add(10);
17        arrayList.Add("Hello");
18        arrayList.Add(3.14);
19        arrayList.Add(true);
20        arrayList.Add("World");
21
22        Console.WriteLine("ArrayList elements:");
23        foreach (object item in arrayList)
24        {
25            Console.WriteLine($"{item} (Type: {item.
               GetType().Name})");
26        }
27
28        Console.WriteLine($"Count: {arrayList.Count}
           ");
29        Console.WriteLine($"Contains 'Hello': {
           arrayList.Contains("Hello")}");
30
31        // Hashtable Demo
32        Console.WriteLine("\n--- Hashtable Demo ---
           ");
33        Hashtable hashtable = new Hashtable();
34
35        hashtable.Add("name", "Ram Sharma");
36        hashtable.Add("age", 25);
37        hashtable.Add("city", "Kathmandu");
38        hashtable.Add("phone", "9841234567");
39
40        Console.WriteLine("Hashtable elements:");
41        foreach (DictionaryEntry entry in hashtable)
42        {
43            Console.WriteLine($"{entry.Key}: {entry.
               Value}");
```

```
44        }
45
46        Console.WriteLine($"Name: {hashtable["name"]
           }");
47
48        // Stack (Non-Generic) Demo
49        Console.WriteLine("\n--- Stack (Non-Generic)
           Demo ---");
50        Stack stack = new Stack();
51
52        stack.Push("First");
53        stack.Push("Second");
54        stack.Push("Third");
55        stack.Push(100);
56
57        Console.WriteLine("Stack elements (LIFO):");
58        while (stack.Count > 0)
59        {
60            Console.WriteLine(stack.Pop());
61        }
62
63        // Queue (Non-Generic) Demo
64        Console.WriteLine("\n--- Queue (Non-Generic)
           Demo ---");
65        Queue queue = new Queue();
66
67        queue.Enqueue("First");
68        queue.Enqueue("Second");
69        queue.Enqueue("Third");
70        queue.Enqueue(200);
71
72        Console.WriteLine("Queue elements (FIFO):");
73        while (queue.Count > 0)
74        {
75            Console.WriteLine(queue.Dequeue());
76        }
77
78        // b) GENERIC COLLECTIONS
79        Console.WriteLine("\n\n=== GENERIC
           COLLECTIONS ===\n");
80
81        // List<T> Demo
82        Console.WriteLine("--- List<T> Demo ---");
83        List<int> numbers = new List<int>();
84
85        numbers.Add(10);
86        numbers.Add(20);
87        numbers.Add(30);
88        numbers.Add(40);
89        numbers.Add(50);
90
```



```

91 Console.WriteLine("List<int> elements:");
92 foreach (int num in numbers)
93 {
94     Console.WriteLine(num);
95 }
96
97 numbers.Remove(30);
98 Console.WriteLine($"After removing 30, Count
99 : {numbers.Count}");
100
101 List<string> names = new List<string>
102 { "Ram", "Sita", "Hari", "Gita" };
103 Console.WriteLine("\nList<string> elements:");
104
105 names.ForEach(name => Console.WriteLine(name
106 ));
107
108 // Dictionary<TKey, TValue> Demo
109 Console.WriteLine("\n--- Dictionary<TKey,
110 TValue> Demo ---");
111 Dictionary<string, string> dictionary = new
112 Dictionary<string, string>();
113
114 dictionary.Add("NPL", "Nepal");
115 dictionary.Add("IND", "India");
116 dictionary.Add("USA", "United States");
117 dictionary.Add("UK", "United Kingdom");
118
119 Console.WriteLine("Dictionary elements:");
120 foreach (KeyValuePair<string, string> kvp in
121 dictionary)
122 {
123     Console.WriteLine($"{kvp.Key}: {kvp.
124 Value}");
125 }
126
127 Console.WriteLine($"NPL stands for: {
128 dictionary["NPL"]}");
129
130 // Dictionary with int key
131 Dictionary<int, string> studentDict = new
132 Dictionary<int, string>();
133 studentDict.Add(101, "Ram Kumar");
134 studentDict.Add(102, "Sita Devi");
135 studentDict.Add(103, "Hari Prasad");
136
137 Console.WriteLine("\nStudent Dictionary:");
138 foreach (var student in studentDict)
139 {
140     Console.WriteLine($"Roll {student.Key}:
141 {student.Value}");
142 }
143
144 // Stack<T> Demo
145 Console.WriteLine("\n--- Stack<T> (Generic)
146 Demo ---");
147 Stack<string> genericStack = new Stack<
148 string>();
149
150 genericStack.Push("Page 1");
151 genericStack.Push("Page 2");
152
153 genericStack.Push("Page 3");
154 genericStack.Push("Page 4");
155
156 Console.WriteLine("Stack<string> elements (
157 LIFO):");
158 Console.WriteLine($"Peek: {genericStack.Peek
159 ()}");
160 while (genericStack.Count > 0)
161 {
162     Console.WriteLine(genericStack.Pop());
163 }
164
165 // Queue<T> Demo
166 Console.WriteLine("\n--- Queue<T> (Generic)
167 Demo ---");
168 Queue<int> genericQueue = new Queue<int>();
169
170 genericQueue.Enqueue(100);
171 genericQueue.Enqueue(200);
172 genericQueue.Enqueue(300);
173 genericQueue.Enqueue(400);
174
175 Console.WriteLine("Queue<int> elements (FIFO
176 ):");
177 Console.WriteLine($"Peek: {genericQueue.Peek
178 ()}");
179 while (genericQueue.Count > 0)
180 {
181     Console.WriteLine(genericQueue.Dequeue()
182 );
183 }
184
185 // HashSet<T> Demo
186 Console.WriteLine("\n--- HashSet<T> Demo ---
187 ");
188 HashSet<int> hashSet = new HashSet<int>();
189
190 hashSet.Add(10);
191 hashSet.Add(20);
192 hashSet.Add(30);
193 hashSet.Add(20); // Duplicate, won't be
194 added
195 hashSet.Add(40);
196
197 Console.WriteLine("HashSet<int> elements (no
198 duplicates):");
199 foreach (int item in hashSet)
200 {
201     Console.WriteLine(item);
202 }
203 Console.WriteLine($"Count: {hashSet.Count}");
204
205 Console.WriteLine("\nLab No.: 10");
206 Console.WriteLine("Name: Samir Paudel");
207 Console.WriteLine("Roll No./Section:
208 114-079/D");
209 }
210 }

```

Output

```

[sam@arch Labs]$ dotnet Lab10.cs
=== NON-GENERIC COLLECTIONS ===

--- ArrayList ---
10 (Int32)
Hello (String)
3.14 (Double)

--- Hashtable ---
name: Ram
age: 25
city: Kathmandu

--- Stack ---
Third
Second
First

--- Queue ---
A
B
C

=== GENERIC COLLECTIONS ===

--- List<T> ---
10
20
30
40

[sam@arch Labs]$ dotnet Lab10.cs
30
40
Names: Ram, Sita, Hari

--- Dictionary<TKey, TValue> ---
NPL: Nepal
IND: India
USA: United States

--- Stack<T> ---
Page3
Page2
Page1

--- Queue<T> ---
100
200
300

--- HashSet<T> ---
10
20
30
Count: 3

Lab No.: 10
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 11: Generic Class with Generic Field and Method

Title

Program to demonstrate the use of Generic Class with Generic field and method.

Theory

Generic Classes use type parameters (T) to create type-safe, reusable code without boxing/unboxing overhead. They can have generic fields, properties, and methods.

Type Constraints restrict generic type parameters using the **where** keyword. Common constraints: **struct** (value types), **class** (reference types), **new()** (parameterless constructor).

Benefits: Type safety at compile time, code reusability, better performance, and support for multiple type parameters.

Code

```
1 using System;
2
3 // Generic Class with Generic Field and Method
4 class Box<T>
5 {
6     // Generic field
7     private T item;
8
9     public void SetItem(T value)
10    {
11        item = value;
12        Console.WriteLine($"Item set: {item}");
13    }
14
15    public T GetItem()
16    {
17        return item;
18    }
19
20    // Generic method
21    public void Display<U>(U data)
22    {
23        Console.WriteLine($"Generic Method - Type: {
24            typeof(U).Name}, Value: {data}");
25    }
26 }
27
28 // Generic class with multiple type parameters
29 class Pair<T1, T2>
30 {
31     public T1 First { get; set; }
32     public T2 Second { get; set; }
33
34     public Pair(T1 first, T2 second)
35     {
36         First = first;
37         Second = second;
38     }
39
40     public void DisplayPair()
41     {
42         Console.WriteLine($"Pair: ({First}, {Second})");
43     }
44
45     // Generic method with constraint
46     public void Swap<T>(ref T a, ref T b)
47     {
48         T temp = a;
49         a = b;
50         b = temp;
51     }
52 }
53
54 // Generic class with constraints
55 class Calculator<T> where T : struct
56 {
57     public void Add(T a, T b)
58     {
59         dynamic x = a;
60         dynamic y = b;
61
62         Console.WriteLine($"{{a}} + {{b}} = {{x + y}}");
63     }
64 }
65
66 class Program
67 {
68     static void Main()
69     {
70         Console.WriteLine("=== Generic Class with
71             Generic Field ===");
72         Box<int> intBox = new Box<int>();
73         intBox.SetItem(100);
74         Console.WriteLine($"Retrieved: {intBox.GetItem()
75             }");
76
77         Console.WriteLine();
78         Box<string> strBox = new Box<string>();
79         strBox.SetItem("Hello Generic");
80         Console.WriteLine($"Retrieved: {strBox.GetItem()
81             }");
82
83         Console.WriteLine("\n=== Generic Method ===");
84         intBox.Display(42);
85         intBox.Display("String value");
86         intBox.Display(3.14);
87
88         Console.WriteLine("\n=== Generic Class with
89             Multiple Parameters ===");
90         Pair<int, string> pair1 = new Pair<int, string>
91             (1, "One");
92         pair1.DisplayPair();
93
94         Pair<string, double> pair2 = new Pair<string,
95             double>("PI", 3.14159);
96         pair2.DisplayPair();
97
98         Console.WriteLine("\n=== Generic Method with
99             Swap ===");
100        int a = 10, b = 20;
101        Console.WriteLine($"Before: a={{a}}, b={{b}}");
102        pair1.Swap(ref a, ref b);
103        Console.WriteLine($"After: a={{a}}, b={{b}}");
104
105        Console.WriteLine("\n=== Generic Class with
106            Constraints ===");
107        Calculator<int> calc = new Calculator<int>();
108        calc.Add(15, 25);
109
110        Calculator<double> calcDouble = new Calculator<
111            double>();
112        calcDouble.Add(5.5, 4.5);
113
114        Console.WriteLine("\nLab No.: 11");
115        Console.WriteLine("Name: Samir Paudel");
116        Console.WriteLine("Roll No./Section: 114-079/D");
117    }
118 }
```

Output

```
[sam@arch Labs]$ dotnet Lab11.cs

=== Generic Class with Generic Field ===
Item set: 100
Retrieved: 100

Item set: Hello Generic
Retrieved: Hello Generic

=== Generic Method ===
Generic Method - Type: Int32, Value: 42
Generic Method - Type: String, Value: String value
Generic Method - Type: Double, Value: 3.14

=== Generic Class with Multiple Parameters ===
Pair: (1, One)
Pair: (PI, 3.14159)

=== Generic Method with Swap ===
Before: a=10, b=20
After: a=20, b=10

=== Generic Class with Constraints ===
15 + 25 = 40
5.5 + 4.5 = 10

Lab No.: 11
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$
```

Lab 12: File Input/Output

Title

WAP to take input from keyboard and write them to a file

Theory

File I/O in C# uses the `System.IO` namespace. `StreamWriter` writes text to files, `StreamReader` reads from files. The `using` statement ensures proper resource disposal.

`File` class provides static methods like `ReadAllText()`, `WriteAllText()`. `StreamWriter` constructor with second parameter `true` enables append mode.

Best practice: Use `using` blocks for automatic file closure and resource cleanup.

Code

```
1 using System;
2 using System.IO;
3
4 class Program
5 {
6     static void Main()
7     {
8         Console.WriteLine("=== Write Input to File ===\n");
9
10        string fileName = "student_data.txt";
11
12        try
13        {
14            // Take input from keyboard
15            Console.Write("Enter your name: ");
16            string name = Console.ReadLine();
17
18            Console.Write("Enter your roll number: ");
19            string rollNo = Console.ReadLine();
20
21            Console.Write("Enter your age: ");
22            string age = Console.ReadLine();
23
24            Console.Write("Enter your address: ");
25            string address = Console.ReadLine();
26
27            // Write to file
28            using (StreamWriter writer = new StreamWriter(
29                fileName))
30            {
31                writer.WriteLine("Student Information");
32                writer.WriteLine("=====");
33                writer.WriteLine($"Name: {name}");
34                writer.WriteLine($"Roll No: {rollNo}");
35                writer.WriteLine($"Age: {age}");
36                writer.WriteLine($"Address: {address}");
37                writer.WriteLine();
38                writer.WriteLine("Lab No.: 12");
39                writer.WriteLine("Name: Samir Paudel");
40                writer.WriteLine("Roll No./Section: 114-079/D");
41            }
42
43            Console.WriteLine($"Data written to {
44                fileName} successfully!");
45
46            // Read and display file contents
47            Console.WriteLine("\n=== File Contents ===\n");
48            ;
49            using (StreamReader reader = new StreamReader(
50                fileName))
51            {
52                string content = reader.ReadToEnd();
53                Console.WriteLine(content);
54            }
55
56            // Append more data
57            Console.WriteLine("\n=== Append Mode ===");
58            Console.Write("\nEnter additional comment: ");
59            string comment = Console.ReadLine();
60        }
61    }
62 }
```

```

57     using (StreamWriter writer = new StreamWriter(
58         fileName, true))
59     {
60         writer.WriteLine($"\\nComment: {comment}");
61         writer.WriteLine($"Date: {DateTime.Now}");
62     }
63
64     Console.WriteLine("Comment appended
65         successfully!");
66
67     // Display final contents
68     Console.WriteLine("\\n=== Final File Contents
69         ===\\n");
70     string finalContent = File.ReadAllText(
71         fileName);

```

```

68     Console.WriteLine(finalContent);
69 }
70 catch (Exception ex)
71 {
72     Console.WriteLine($"Error: {ex.Message}");
73 }
74
75 Console.WriteLine("\\nLab No.: 12");
76 Console.WriteLine("Name: Samir Paudel");
77 Console.WriteLine("Roll No./Section: 114-079/D")
78 ;
79 }

```

Output

Labs > student_data.txt

```

1  Student Information
2  =====
3  Name: Samir
4  Roll No: 114/079
5  Age: 22
6  Address: Lalitpur
7
8  Lab No.: 12
9  Name: Samir Paudel
10 Roll No./Section: 114-079/D
11
12 Comment: Nirvana
13 Date: 12/19/2025 8:49:20 AM
14

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS ... bash - Labs

[sam@arch Labs]\$ dotnet Lab12.cs

Enter your age: 22

Enter your address: Lalitpur

Data written to student_data.txt successfully!

=== File Contents ===

```

Student Information
=====
Name: Samir
Roll No: 114/079
Age: 22
Address: Lalitpur

```

1

Lab 13: LINQ (Language Integrated Query)

Title

WAP to demonstrate the concept of LINQ

Theory

LINQ (Language Integrated Query) provides query capabilities directly in C#. It works with collections, databases, XML, and more.

Key operations: Where (filter), Select (project), OrderBy (sort), GroupBy (group), aggregates (Sum, Average, Count, Max, Min).

Two syntaxes: Query syntax (SQL-like) and Method syntax (using lambda expressions). Both produce same results.

Code

```
1 using System;
2 using System.Linq;
3 using System.Collections.Generic;
4
5 class Student
6 {
7     public int RollNo { get; set; }
8     public string Name { get; set; }
9     public int Age { get; set; }
10    public double Marks { get; set; }
11    public string City { get; set; }
12 }
13
14 class Program
15 {
16     static void Main()
17     {
18         // Sample data
19         List<Student> students = new List<Student>
20         {
21             new Student { RollNo = 1, Name = "Ram", Age =
22                 20, Marks = 85, City = "Kathmandu" },
23             new Student { RollNo = 2, Name = "Sita", Age =
24                 19, Marks = 92, City = "Pokhara" },
25             new Student { RollNo = 3, Name = "Hari", Age =
26                 21, Marks = 78, City = "Kathmandu" },
27             new Student { RollNo = 4, Name = "Gita", Age =
28                 20, Marks = 88, City = "Lalitpur" },
29             new Student { RollNo = 5, Name = "Shyam", Age =
30                 22, Marks = 95, City = "Pokhara" }
31         };
32
33         int[] numbers = { 5, 2, 8, 1, 9, 3, 7, 4, 6 };
34
35         Console.WriteLine("=== LINQ - Where (Filtering) ===");
36         var passedStudents = students.Where(s => s.Marks
37             >= 80);
38         foreach (var s in passedStudents)
39             Console.WriteLine($"{s.Name}: {s.Marks}");
40
41         Console.WriteLine("\n=== LINQ - Select (Projection) ===");
42         var studentNames = students.Select(s => s.Name);
43         Console.WriteLine("Names: " + string.Join(", ",
44             studentNames));
45
46         Console.WriteLine("\n=== LINQ - OrderBy ===");
47         var sortedByMarks = students.OrderByDescending(s
48             => s.Marks);
49         foreach (var s in sortedByMarks)
50             Console.WriteLine($"{s.Name}: {s.Marks}");
51
52         Console.WriteLine("\n=== LINQ - GroupBy ===");
53         var studentsByCity = students.GroupBy(s => s.
54             City);
55         foreach (var group in studentsByCity)
56         {
57             Console.WriteLine($"{group.Key}:");
58             foreach (var s in group)
59                 Console.WriteLine($"    {s.Name}");
60         }
61
62         Console.WriteLine("\n=== LINQ - Aggregate Functions ===");
63         Console.WriteLine($"Total Students: {students.
64             Count()}");
65         Console.WriteLine($"Average Marks: {students.
66             Average(s => s.Marks):F2}");
67         Console.WriteLine($"Highest Marks: {students.Max(
68             s => s.Marks)}");
69         Console.WriteLine($"Lowest Marks: {students.Min(
70             s => s.Marks)}");
71         Console.WriteLine($"Sum of Marks: {students.Sum(
72             s => s.Marks)}");
73
74         Console.WriteLine("\n=== LINQ - First, Last, Single ===");
75         var firstStudent = students.First();
76         Console.WriteLine($"First: {firstStudent.Name}");
77
78         var topStudent = students.First(s => s.Marks >
79             90);
80         Console.WriteLine($"First with >90: {topStudent.
81             Name}");
82
83         Console.WriteLine("\n=== LINQ - Query Syntax ===");
84         var query = from s in students
85                     where s.Age >= 20 && s.Marks >= 85
86                     orderby s.Marks descending
87                     select new { s.Name, s.Marks, s.City };
88
89         foreach (var item in query)
90             Console.WriteLine($"{item.Name} ({item.City}):
91                 {item.Marks}");
92
93         Console.WriteLine("\n=== LINQ on Arrays ===");
94         var evenNumbers = numbers.Where(n => n % 2 == 0);
95         Console.WriteLine("Even: " + string.Join(", ",
96             evenNumbers));
97
98         var sortedNumbers = numbers.OrderBy(n => n);
99         Console.WriteLine("Sorted: " + string.Join(", ",
100             sortedNumbers));
101
102         Console.WriteLine("\n=== LINQ - Any, All ===");
103         Console.WriteLine($"Any student with 100? {
104             students.Any(s => s.Marks == 100)}");
105         Console.WriteLine($"All students passed (>50)? {
106             students.All(s => s.Marks > 50)}");
107
108         Console.WriteLine("\nLab No.: 13");
109         Console.WriteLine("Name: Samir Paudel");
110         Console.WriteLine("Roll No./Section: 114-079/D")
111     }
112 }
```

Output

```
[sam@arch Labs]$ dotnet Lab13.cs
=== LINQ - Where (Filtering) ===
Ram: 85
Sita: 92
Gita: 88
Shyam: 95

=== LINQ - Select (Projection) ===
Names: Ram, Sita, Hari, Gita, Shyam

=== LINQ - OrderBy ===
Shyam: 95
Sita: 92
Gita: 88
Ram: 85
Hari: 78

=== LINQ - GroupBy ===
Kathmandu:
  Ram
  Hari
Pokhara:
  Sita
  Shyam

[sam@arch Labs]$ dotnet Lab13.cs
=== LINQ - Aggregate Functions ===
Total Students: 5
Average Marks: 87.60
Highest Marks: 95
Lowest Marks: 78
Sum of Marks: 438

=== LINQ - First, Last, Single ===
First: Ram
First with >90: Sita

=== LINQ - Query Syntax ===
Shyam (Pokhara): 95
Gita (Lalitpur): 88
Ram (Kathmandu): 85

=== LINQ on Arrays ===
Even: 2, 8, 4, 6
Sorted: 1, 2, 3, 4, 5, 6, 7, 8, 9

=== LINQ - Any, All ===
Any student with 100? False
All students passed (>50)? True

Lab No.: 13

[sam@arch Labs]$ dotnet Lab13.cs
Highest Marks: 95
Lowest Marks: 78
Sum of Marks: 438

=== LINQ - First, Last, Single ===
First: Ram
First with >90: Sita

=== LINQ - Query Syntax ===
Shyam (Pokhara): 95
Gita (Lalitpur): 88
Ram (Kathmandu): 85

=== LINQ on Arrays ===
Even: 2, 8, 4, 6
Sorted: 1, 2, 3, 4, 5, 6, 7, 8, 9

=== LINQ - Any, All ===
Any student with 100? False
All students passed (>50)? True

Lab No.: 13
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$
```

Lab 14: Lambda Expressions

Title

WAP to demonstrate Lambda Expressions in C#.

Theory

Lambda Expressions are anonymous functions using the `=>` operator (read as “goes to”). Syntax: (parameters) `=>` expression or (parameters) `=>` { statements }.

Commonly used with LINQ, delegates (Func, Action), and event handlers. Provide concise way to write inline functions.

Types: Expression lambdas (single expression) and Statement lambdas (code block with braces).

Code

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4
5 class Program
6 {
7     static void Main()
8     {
9         Console.WriteLine("=== Lambda Expression - Basic ===");
10
11         // Lambda with no parameters
12         Action greet = () => Console.WriteLine("Hello
13             from Lambda!");
14         greet();
15
16         // Lambda with one parameter
17         Action<string> sayHello = name => Console.
18             WriteLine($"Hello, {name}!");
19         sayHello("Ram");
20
21         // Lambda with multiple parameters
22         Func<int, int, int> add = (a, b) => a + b;
23         Console.WriteLine($"10 + 5 = {add(10, 5)}");
24
25         Func<int, int, int> multiply = (x, y) => x * y;
26         Console.WriteLine($"10 * 5 = {multiply(10, 5)}");
27
28         Console.WriteLine("\n=== Lambda with Collections ===");
29         List<int> numbers = new List<int> { 1, 2, 3, 4,
30             5, 6, 7, 8, 9, 10 };
31
32         // Filter using lambda
33         var evenNumbers = numbers.Where(n => n % 2 == 0)
34             .ToList();
35         Console.WriteLine("Even: " + string.Join(", ",
36             evenNumbers));
37
38         // Transform using lambda
39         var squares = numbers.Select(n => n * n).ToList()
40             ();
41         Console.WriteLine("Squares: " + string.Join(", ",
42             squares));
43
44         // Sort using lambda
45         List<string> names = new List<string> { "Ram", "
46             Hari", "Sita", "Gita" };
47         var sortedNames = names.OrderBy(n => n.Length).
48             ToList();
49         Console.WriteLine("Sorted by length: " + string.
50             Join(", ", sortedNames));
51
52         Console.WriteLine("\n=== Lambda for Filtering ===");
53         var greaterThan5 = numbers.Where(n => n > 5).
54             ToList();
55         Console.WriteLine("Greater than 5: " + string.
56             Join(", ", greaterThan5));
57
58         Console.WriteLine("\n=== Lambda with Objects ===");
59         List<Person> people = new List<Person>
60         {
61             new Person { Name = "Ram", Age = 25, Salary =
62                 50000 },
63             new Person { Name = "Sita", Age = 30, Salary =
64                 60000 },
65             new Person { Name = "Hari", Age = 22, Salary =
66                 45000 },
67             new Person { Name = "Gita", Age = 28, Salary =
68                 55000 }
69         };
70
71         // Filter
72         var youngPeople = people.Where(p => p.Age < 28).
73             ToList();
74         Console.WriteLine("\nAge < 28:");
75         youngPeople.ForEach(p => Console.WriteLine($"{p.
76             Name}: {p.Age}"));
77
78         // Select specific properties
79         var salaries = people.Select(p => p.Salary).
80             ToList();
81         Console.WriteLine("\nSalaries: " + string.Join("
82             ", ", salaries));
83
84         // Order by
85         var orderedBySalary = people.OrderByDescending(p
86             => p.Salary).ToList();
87         Console.WriteLine("\nOrdered by Salary:");
88         orderedBySalary.ForEach(p => Console.WriteLine($"
89             {p.Name}: {p.Salary}"));
90
91         Console.WriteLine("\n=== Lambda - Multiple
92             Statements ===");
93         Func<int, int, string> compare = (a, b) =>
94         {
95             if (a > b) return $"{a} is greater";
96             else if (a < b) return $"{b} is greater";
97             else return "Both are equal";
98         };
99         Console.WriteLine(compare(10, 5));
100        Console.WriteLine(compare(3, 8));
101
102        Console.WriteLine("\n=== Lambda - Aggregate
103            Operations ===");
104        int sum = numbers.Sum();
105        double average = numbers.Average();
106        int max = numbers.Max();
107        int min = numbers.Min();
108
109        Console.WriteLine($"Sum: {sum}");
110        Console.WriteLine($"Average: {average}");
111        Console.WriteLine($"Max: {max}");
112        Console.WriteLine($"Min: {min}");
113
114        // Custom aggregate
115        var totalSalary = people.Sum(p => p.Salary);
116        Console.WriteLine($"Total Salary: {totalSalary}");
117
118        Console.WriteLine("\n=== Lambda - Find ===");
119        var firstHighEarner = people.FirstOrDefault(p =>
120            p.Salary > 55000);
121        if (firstHighEarner != null)
122            Console.WriteLine($"First high earner: {
123                firstHighEarner.Name}");
```

```

98 Console.WriteLine("\nLab No.: 14");
99 Console.WriteLine("Name: Samir Paudel");
100 Console.WriteLine("Roll No./Section: 114-079/D");
101 ;
102 }
103 }
104

```

```

105 class Person
106 {
107     public string Name { get; set; }
108     public int Age { get; set; }
109     public double Salary { get; set; }
110 }

```

Output

```
[sam@arch Labs]$ dotnet Lab14.cs
```

```

Hello from Lambda!
Hello, Ram!
10 + 5 = 15
10 * 5 = 50

=== Lambda with Collections ===
Even: 2, 4, 6, 8, 10
Squares: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100
Sorted by length: Ram, Hari, Sita, Gita

=== Lambda for Filtering ===
Greater than 5: 6, 7, 8, 9, 10

=== Lambda with Objects ===

Age < 28:
Ram: 25
Hari: 22

Salaries: 50000, 60000, 45000, 55000

Ordered by Salary:
Sita: 60000
Gita: 55000
Ram: 50000
Hari: 45000

=== Lambda - Multiple Statements ===

```

```
[sam@arch Labs]$ dotnet Lab14.cs
```

```

Salaries: 50000, 60000, 45000, 55000

Ordered by Salary:
Sita: 60000
Gita: 55000
Ram: 50000
Hari: 45000

=== Lambda - Multiple Statements ===
10 is greater
8 is greater

=== Lambda - Aggregate Operations ===
Sum: 55
Average: 5.5
Max: 10
Min: 1

Total Salary: 210000

=== Lambda - Find ===
First high earner: Sita

Lab No.: 14
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```


Lab 15: Exception Handling

Title

WAP to:

- demonstrate exception handling in C# using try, catch and finally blocks
- deal with throw keyword in exception handling
- demonstrate custom exception handling

Theory

Exception Handling uses try-catch-finally blocks. **try** contains code that may throw exceptions. **catch** handles specific exception types. **finally** executes regardless of exception occurrence.

throw keyword raises exceptions explicitly. Custom exceptions inherit from **Exception** class and can have custom properties for additional information.

Multiple catch blocks handle different exception types. Specific exceptions should be caught before general ones.

Code

```
1 using System;
2
3 // Custom Exception
4 class InvalidAgeException : Exception
5 {
6     public InvalidAgeException(string message) : base(
7         message) { }
8 }
9
10 class InsufficientBalanceException : Exception
11 {
12     public double Balance { get; set; }
13     public double RequestedAmount { get; set; }
14
15     public InsufficientBalanceException(string message
16         , double balance, double requested)
17         : base(message)
18     {
19         Balance = balance;
20         RequestedAmount = requested;
21     }
22 }
23
24 class BankAccount
25 {
26     private double balance;
27
28     public BankAccount(double initialBalance)
29     {
30         balance = initialBalance;
31     }
32
33     public void Withdraw(double amount)
34     {
35         if (amount > balance)
36             throw new InsufficientBalanceException(
37                 "Insufficient balance!", balance, amount);
38
39         balance -= amount;
40         Console.WriteLine($"Withdrawn: {amount}, Balance
41             : {balance}");
42     }
43 }
44
45 class Program
46 {
47     static void Main()
48     {
49         Console.WriteLine("=== Try-Catch-Finally ===\n");
50
51         try
52         {
53             Console.Write("Enter first number: ");
54             int num1 = int.Parse(Console.ReadLine());
55
56             Console.Write("Enter second number: ");
57             int num2 = int.Parse(Console.ReadLine());
58
59             int result = num1 / num2;
60
61             Console.WriteLine($"Result: {num1} / {num2} =
62                 {result}");
63
64             catch (DivideByZeroException ex)
65             {
66                 Console.WriteLine($"Error: {ex.Message}");
67             }
68
69             catch (FormatException ex)
70             {
71                 Console.WriteLine($"Invalid input: {ex.Message}");
72             }
73
74             catch (Exception ex)
75             {
76                 Console.WriteLine($"General Error: {ex.Message}");
77             }
78
79             finally
80             {
81                 Console.WriteLine("Finally block executed -
82                     Cleanup done");
83             }
84
85             Console.WriteLine("\n=== Throw Keyword ===\n");
86
87             try
88             {
89                 ValidateAge(15);
90             }
91             catch (InvalidAgeException ex)
92             {
93                 Console.WriteLine($"Custom Exception: {ex.
94                     Message}");
95             }
96
97             Console.WriteLine("\n=== Multiple Catch Blocks
98                 ===\n");
99
100             try
101             {
102                 int[] numbers = { 1, 2, 3 };
103                 Console.WriteLine(numbers[5]); // Index out of
104                     range
105             }
106             catch (IndexOutOfRangeException ex)
107             {
108                 Console.WriteLine($"Index Error: {ex.Message}");
109             }
110
111             catch (Exception ex)
112             {
113                 Console.WriteLine($"Error: {ex.Message}");
114             }
115
116             Console.WriteLine("\n=== Custom Exception with
117                 Properties ===\n");
118
119             BankAccount account = new BankAccount(5000);
120
121             try
122             {
123 
```

<pre> 109 account.Withdraw(3000); 110 account.Withdraw(3000); // This will throw exception 111 } 112 catch (InsufficientBalanceException ex) 113 { 114 Console.WriteLine(\$"Error: {ex.Message}"); 115 Console.WriteLine(\$"Balance: {ex.Balance}"); 116 Console.WriteLine(\$"Requested: {ex. RequestedAmount}"); 117 Console.WriteLine(\$"Shortage: {ex. RequestedAmount - ex.Balance}"); 118 } 119 120 Console.WriteLine("\n=== Nested Try-Catch ===\n"); 121 122 try 123 { 124 try 125 { 126 int x = 10 / 0; 127 } 128 catch (DivideByZeroException) 129 { 130 Console.WriteLine("Inner catch: Division by zero"); 131 throw; // Re-throw the exception 132 } 133 } 134 catch (Exception ex) 135 { 136 Console.WriteLine(\$"Outer catch: {ex.GetType() .Name}"); 137 } 138 139 Console.WriteLine("\n=== Exception with Stack Trace ===\n"); 140 141 try 142 { </pre>	<pre> 143 Method1(); 144 } 145 catch (Exception ex) 146 { 147 Console.WriteLine(\$"Exception: {ex.Message}"); 148 Console.WriteLine(\$"Source: {ex.Source}"); 149 Console.WriteLine("Stack Trace:"); 150 Console.WriteLine(ex.StackTrace); 151 } 152 153 Console.WriteLine("\nLab No.: 15"); 154 Console.WriteLine("Name: Samir Paudel"); 155 Console.WriteLine("Roll No./Section: 114-079/D"); 156 } 157 158 static void ValidateAge(int age) 159 { 160 if (age < 18) 161 throw new InvalidAgeException(\$"Age {age} is below 18!"); 162 163 Console.WriteLine(\$"Age {age} is valid"); 164 } 165 166 static void Method1() 167 { 168 Method2(); 169 } 170 171 static void Method2() 172 { 173 Method3(); 174 } 175 176 static void Method3() 177 { 178 throw new Exception("Error in Method3"); 179 } 180 } </pre>
--	--

Output

<pre> [sam@arch Labs]\$ echo -e "10\n2" dotnet Lab15.cs === Try-Catch-Finally === Enter first number: Enter second number: Result: 10 / 2 = 5 Finally block executed - Cleanup done === Throw Keyword === Custom Exception: Age 15 is below 18! === Multiple Catch Blocks === Index Error: Index was outside the bounds of the array. === Custom Exception with Properties === Withdrawn: 3000, Balance: 2000 Error: Insufficient balance! Balance: 2000 Requested: 3000 Shortage: 1000 === Nested Try-Catch === Inner catch: Division by zero Outer catch: DivideByZeroException </pre>	<pre> [sam@arch Labs]\$ dotnet Lab15.cs Requested: 3000 Shortage: 1000 === Nested Try-Catch === Inner catch: Division by zero Outer catch: DivideByZeroException === Exception with Stack Trace === Exception: Error in Method3 Source: Lab15 Stack Trace: at Program.Method3() in /home/sam/Github/semester-work s/semester-6/dotnet/Labs/Lab15.cs:line 179 at Program.Method2() in /home/sam/Github/semester-work s/semester-6/dotnet/Labs/Lab15.cs:line 174 at Program.Method1() in /home/sam/Github/semester-work s/semester-6/dotnet/Labs/Lab15.cs:line 169 at Program.Main() in /home/sam/Github/semester-works/s emester-6/dotnet/Labs/Lab15.cs:line 144 Lab No.: 15 Name: Samir Paudel Roll No./Section: 114-079/D [sam@arch Labs]\$ </pre>
---	--

Lab 16: Attributes in C#

Title

WAP to:

- a) use built-in attributes in C#
- b) create and use custom attribute in C#

Theory

Attributes add metadata to code elements (classes, methods, properties). They don't affect program logic but provide information for reflection, serialization, etc.

Built-in attributes: [Obsolete], [Serializable], [Conditional]. Custom attributes inherit from Attribute class.

AttributeUsage controls where attributes can be applied. Reflection (GetCustomAttributes) reads attribute data at runtime.

Code

```
1 using System;
2
3 // Custom Attribute
4 [AttributeUsage(AttributeTargets.Class |
5     AttributeTargets.Method, AllowMultiple = true)]
6 class DeveloperInfoAttribute : Attribute
7 {
8     public string Name { get; set; }
9     public string Date { get; set; }
10    public string Version { get; set; }
11
12    public DeveloperInfoAttribute(string name)
13    {
14        Name = name;
15        Date = DateTime.Now.ToShortDateString();
16    }
17
18    public void DisplayInfo()
19    {
20        Console.WriteLine($"Developer: {Name}");
21        Console.WriteLine($"Date: {Date}");
22        Console.WriteLine($"Version: {Version} ?? "1.0"");
23    }
24 }
25
26 [AttributeUsage(AttributeTargets.Method)]
27 class TestMethodAttribute : Attribute
28 {
29     public string Description { get; set; }
30
31     public TestMethodAttribute(string description)
32     {
33         Description = description;
34     }
35 }
36
37 // Using Built-in Attributes
38 [Serializable]
39 [DeveloperInfo("Samir Paudel", Version = "2.0")]
40 class Student
41 {
42     public string Name { get; set; }
43     public int Age { get; set; }
44
45     [Obsolete("Use GetFullInfo() instead")]
46     public void DisplayInfo()
47     {
48         Console.WriteLine($"Name: {Name}, Age: {Age}");
49     }
50
51     [DeveloperInfo("Samir", Version = "1.5")]
52     [TestMethod("Tests full information display")]
53     public void GetFullInfo()
54     {
55         Console.WriteLine($"Student: {Name}, Age: {Age}");
56     }
57 }
58
59 class Calculator
```

```
59 {
60     [Obsolete("This method is deprecated. Use
61         AddNumbers instead.", true)]
62     public int Add(int a, int b)
63     {
64         return a + b;
65     }
66
67     public int AddNumbers(int a, int b)
68     {
69         return a + b;
70     }
71
72     [DeveloperInfo("Ram", Version = "2.1")]
73     [TestMethod("Tests multiplication")]
74     public int Multiply(int a, int b)
75     {
76         return a * b;
77     }
78 }
79
80 class Program
81 {
82     static void Main()
83     {
84         Console.WriteLine("=== Built-in Attributes ===\n");
85
86         Student student = new Student { Name = "Ram",
87             Age = 20 };
88
89         // Using obsolete method (generates warning)
90         student.DisplayInfo();
91
92         // Using new method
93         student.GetFullInfo();
94
95         Console.WriteLine("\n=== Custom Attributes ===\n");
96
97         // Reading custom attributes from class
98         Type studentType = typeof(Student);
99         var classAttributes = studentType.
100             GetCustomAttributes(typeof(
101                 DeveloperInfoAttribute), false);
102
103         Console.WriteLine($"Class: {studentType.Name}");
104         foreach (DeveloperInfoAttribute attr in
105             classAttributes)
106         {
107             attr.DisplayInfo();
108             Console.WriteLine();
109         }
110
111         // Reading attributes from methods
112         var methods = studentType.GetMethods();
113
114         foreach (var method in methods)
115         {
116             var devAttrs = method.GetCustomAttributes(
117                 typeof(DeveloperInfoAttribute), false);
118             var testAttrs = method.GetCustomAttributes(
```

```

113         typeof(TestMethodInfoAttribute), false);
114     if (devAttrs.Length > 0 || testAttrs.Length >
115         0)
116     {
117         Console.WriteLine($"Method: {method.Name}");
118         foreach (DeveloperInfoAttribute attr in
119             devAttrs)
120         {
121             Console.WriteLine($"    {attr.Name}");
122             attr.DisplayInfo();
123         }
124         foreach (TestMethodInfoAttribute attr in
125             testAttrs)
126         {
127             Console.WriteLine($"    Test: {attr.
128                 Description}");
129         }
130         Console.WriteLine();
131     }
132     Console.WriteLine("=== Other Built-in Attributes
133         ===\n");
134     // Conditional attribute example
135     TestConditionalMethod();
136     Console.WriteLine("\n=== Reading All Attributes
137         ===\n");
138     Type calcType = typeof(Calculator);
139     var calcMethods = calcType.GetMethods();
140     foreach (var method in calcMethods)
141     {
142         if (method.DeclaringType == calcType)
143         {
144             Console.WriteLine($"Method: {method.Name}");
145             var attrs = method.GetCustomAttributes(false);
146             foreach (var attr in attrs)
147             {
148                 Console.WriteLine($"    {attr.Name}");
149                 attr.DisplayInfo();
150             }
151         }
152     }
153     Console.WriteLine($"Attribute: {attr.
154         GetType().Name}");
155     if (attr is DeveloperInfoAttribute devAttr)
156     {
157         Console.WriteLine($"    Developer: {
158             devAttr.Name}");
159         Console.WriteLine($"    Version: {
160             devAttr.Version}");
161     }
162     else if (attr is TestMethodInfoAttribute
163         testAttr)
164     {
165         Console.WriteLine($"    Description: {
166             testAttr.Description}");
167     }
168     else if (attr is ObsoleteAttribute obsAttr)
169     {
170         Console.WriteLine($"    Message: {
171             obsAttr.Message}");
172         Console.WriteLine($"    IsError: {
173             obsAttr.IsError}");
174     }
175     }
176     Console.WriteLine("\n=== Serializable Attribute
177         ===");
178     Console.WriteLine($"Is Student Serializable? {
179         studentType.IsSerializable}");
180     Console.WriteLine("\nLab No.: 16");
181     Console.WriteLine($"Name: Samir Paudel");
182     Console.WriteLine($"Roll No./Section: 114-079/D");
183     ;
184     }
185     [System.Diagnostics.Conditional("DEBUG")]
186     static void TestConditionalMethod()
187     {
188         Console.WriteLine("This only executes in DEBUG
189             mode");
190     }
191 }

```

Output

```

[sam@arch Labs]$ dotnet Lab16.cs
is obsolete: 'Formatter-based serialization is obsolete
and should not be used.' (https://aka.ms/dotnet-warnings/
SYSLIB0050)
=== Built-in Attributes ===

Ram, Age: 20
Student: Ram, Age: 20

=== Custom Attributes ===

Class: Student
Developer: Samir Paudel
Date: 12/19/2025
Version: 2.0

Method: GetFullInfo
Developer: Samir
Date: 12/19/2025
Version: 1.5
Test: Tests full information display

=== Other Built-in Attributes ===

This only executes in DEBUG mode

=== Reading All Attributes ===

```

```

[sam@arch Labs]$ dotnet Lab16.cs
=== Reading All Attributes ===

Method: Add
Attribute: ObsoleteAttribute
Message: This method is deprecated. Use AddNumbers in
stead.
IsError: True

Method: AddNumbers

Method: Multiply
Attribute: DeveloperInfoAttribute
Developer: Ram
Version: 2.1
Attribute: TestMethodInfoAttribute
Description: Tests multiplication

=== Serializable Attribute ===
Is Student Serializable? True

Lab No.: 16
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 17: Asynchronous Programming

Title

WAP to demonstrate asynchronous programming in C# using async and await keywords.

Theory

Async/Await enables asynchronous programming without blocking threads. **async** methods return **Task** or **Task<T>**. **await** pauses execution until task completes.

Task.WhenAll waits for all tasks to complete. **Task.WhenAny** waits for first completion. Async improves application responsiveness and scalability.

Best for I/O operations, network calls, file operations. Doesn't create new threads - uses task scheduler efficiently.

Code

```
1 using System;
2 using System.Threading;
3 using System.Threading.Tasks;
4 using System.Net.Http;
5
6 class Program
7 {
8     static async Task Main(string[] args)
9     {
10         Console.WriteLine("=== Async/Await Basics ===\n");
11
12         Console.WriteLine("Starting async operation...")
13         ;
14         await PerformAsyncOperation();
15         Console.WriteLine("Async operation completed!\n");
16
17         Console.WriteLine("=== Multiple Async Tasks ===\n");
18         await RunMultipleTasks();
19
20         Console.WriteLine("\n=== Async with Return Value ===\n");
21         int result = await CalculateAsync(10, 5);
22         Console.WriteLine($"Calculation result: {result}");
23
24         Console.WriteLine("\n=== Parallel Async Tasks ===\n");
25         await RunParallelTasks();
26
27         Console.WriteLine("\n=== Exception Handling in Async ===\n");
28         await HandleAsyncException();
29
30         Console.WriteLine("\n=== Task.WhenAll ===\n");
31         await DemoTaskWhenAll();
32
33         Console.WriteLine("\n=== Task.WhenAny ===\n");
34         await DemoTaskWhenAny();
35
36         Console.WriteLine("\nLab No.: 17");
37         Console.WriteLine("Name: Samir Paudel");
38         Console.WriteLine("Roll No./Section: 114-079/D");
39     }
40
41     static async Task PerformAsyncOperation()
42     {
43         Console.WriteLine(" Task started...");
44         await Task.Delay(2000); // Simulate 2 second
45         delay
46         Console.WriteLine(" Task completed after 2
47         seconds");
48     }
49
50     static async Task RunMultipleTasks()
51     {
52         Console.WriteLine("Task 1 starting...");
53         await Task1();
54
55         Console.WriteLine("Task 2 starting...");
56         await Task2();
57
58         Console.WriteLine("Both tasks completed");
59     }
60
61     static async Task Task1()
62     {
63         await Task.Delay(1000);
64         Console.WriteLine(" Task 1 done (1 sec)");
65     }
66
67     static async Task Task2()
68     {
69         await Task.Delay(1500);
70         Console.WriteLine(" Task 2 done (1.5 sec)");
71     }
72
73     static async Task<int> CalculateAsync(int a, int b)
74     {
75         Console.WriteLine($"Calculating {a} + {b}...");
76         await Task.Delay(1000);
77         return a + b;
78     }
79
80     static async Task RunParallelTasks()
81     {
82         Console.WriteLine("Starting 3 tasks in parallel
83         ...");
84
85         var task1 = DownloadDataAsync("File1", 1000);
86         var task2 = DownloadDataAsync("File2", 1500);
87         var task3 = DownloadDataAsync("File3", 2000);
88
89         await Task.WhenAll(task1, task2, task3);
90
91         Console.WriteLine("\nAll downloads completed!");
92         Console.WriteLine($"Total time: ~2 seconds (
93         parallel execution)");
94     }
95
96     static async Task DownloadDataAsync(string
97         fileName, int delay)
98     {
99         Console.WriteLine($" Downloading {fileName}...")
100         ;
101         await Task.Delay(delay);
102         Console.WriteLine($" {fileName} downloaded ({
103         delay}ms)");
104     }
105
106     static async Task HandleAsyncException()
107     {
108         try
109         {
110             await TaskThatThrowsException();
111         }
112         catch (Exception ex)
113         {
114             Console.WriteLine($"Caught exception: {ex.
115             Message}");
116         }
117     }
118
119     static async Task TaskThatThrowsException()
120     {
121         // This task is designed to throw an exception
122     }
123 }
```

```

112     await Task.Delay(500);
113     throw new Exception("Something went wrong in
114         async task!");
115 }
116 static async Task DemoTaskWhenAll()
117 {
118     var tasks = new[]
119     {
120         ProcessDataAsync("Data1", 1000),
121         ProcessDataAsync("Data2", 1500),
122         ProcessDataAsync("Data3", 800)
123     };
124
125     Console.WriteLine("Waiting for all tasks...");
126     var results = await Task.WhenAll(tasks);
127
128     Console.WriteLine("\nAll results:");
129     foreach (var result in results)
130         Console.WriteLine($" {result}");
131 }
132
133 static async Task<string> ProcessDataAsync(string
134     data, int delay)
135 {
136     await Task.Delay(delay);
137     return $"[{data}] processed in {delay}ms";
138 }
139 static async Task DemoTaskWhenAny()
140 {
141     var task1 = DelayedTaskAsync("Fast Task", 1000);
142     var task2 = DelayedTaskAsync("Medium Task",
143         2000);
144     var task3 = DelayedTaskAsync("Slow Task", 3000);
145
146     Console.WriteLine("Waiting for first task to
147         complete...");
148
149     var completedTask = await Task.WhenAny(task1,
150         task2, task3);
151     var result = await completedTask;
152
153     Console.WriteLine($" \nFirst completed: {result}"
154         );
155     Console.WriteLine("Other tasks still running in
156         background");
157 }
158
159 static async Task<string> DelayedTaskAsync(string
160     name, int delay)
161 {
162     Console.WriteLine($" {name} started");
163     await Task.Delay(delay);
164     Console.WriteLine($" {name} completed");
165     return name;
166 }

```

Output

```
[sam@arch Labs]$ dotnet Lab17.cs
```

```

Starting async operation...
Task started...
Task completed after 2 seconds
Async operation completed!

=== Multiple Async Tasks ===

Task 1 starting...
Task 1 done (1 sec)
Task 2 starting...
Task 2 done (1.5 sec)
Both tasks completed

=== Async with Return Value ===

Calculating 10 + 5...
Calculation result: 15

=== Parallel Async Tasks ===

Starting 3 tasks in parallel...
Downloading File1...
Downloading File2...
Downloading File3...
File1 downloaded (1000ms)

```

```
[sam@arch Labs]$ dotnet Lab17.cs
```

```

Caught exception: Something went wrong in async task!

=== Task.WhenAll ===

Waiting for all tasks...

All results:
Data1 processed in 1000ms
Data2 processed in 1500ms
Data3 processed in 800ms

=== Task.WhenAny ===

Fast Task started
Medium Task started
Slow Task started
Waiting for first task to complete...
Fast Task completed

First completed: Fast Task
Other tasks still running in background

Lab No.: 17
Name: Samir Paudel
Roll No./Section: 114-079/D
[sam@arch Labs]$

```

Lab 18: ASP.NET Core MVC Application

Title

Create an ASP.Net Core MVC project (WebApp1BySamir) with Razor pages, Model validation, and CRUD forms.

Theory

ASP.NET Core MVC follows Model-View-Controller pattern. Models represent data with validation attributes. Views (Razor pages) display UI. Controllers handle HTTP requests and return responses. Tag Helpers simplify form creation. Server-side validation ensures data integrity before processing.

Project Structure

```
WebApp1BySamir/  
  Controllers/  
    MyController.cs  
  Models/  
    Student.cs  
  Views/  
    My/  
      MyRazorPage.cshtml  
      CreateStudent.cshtml  
      StudentDetails.cshtml
```

Key Code

Student Model with Validation:

```
1 public class Student  
2 {  
3     [Required(ErrorMessage = "Student ID is required")]  
4     [Range(1, 9999, ErrorMessage = "ID must be between 1 and 9999")]  
5     public int StdID { get; set; }  
6  
7     [Required, StringLength(100, MinimumLength = 2)]  
8     public string Name { get; set; }  
9  
10    [Required, EmailAddress]  
11    public string Email { get; set; }  
12 }
```

Controller Actions:

```
1 [HttpPost]  
2 public IActionResult CreateStudent(Student student)  
3 {  
4     if (ModelState.IsValid)  
5         return RedirectToAction("StudentDetails", student);  
6     return View(student);  
7 }
```

Features

- MyRazorPage: Displays current date/time, name (Samir Paudel), roll no (114-079/D), multiplication table of 80
- Create Student form with validation (Name, Email, Age, Faculty, etc.)
- Server-side validation with error messages in red
- Redirect to StudentDetails page on successful submission
- Navbar links: "MyRazorPage" and "Create Student Record"

Output

The figure consists of two side-by-side screenshots from a web application. The left screenshot shows a 'Create Student Record' form. It has a green header bar with the title. Below the title, there are input fields for 'Student ID' (value: 1), 'Full Name' (value: Sansi Pseud), 'Sansi Pseud' (value: Sansi Pseud), 'Address' (value: Laffour), 'Faculty' (a dropdown menu), 'Email Address' (value: contact@sanipseud.me), 'Phone Number' (value: 0940253621), and 'Age' (value: 22). There is a green 'Submit' button at the bottom. The right screenshot shows the 'Student Details' page. It has a blue header bar with the title. Below the title, there is a green message box that says 'Student record created successfully!'. Below that, there is a table with the following data: Student ID: 1, Full Name: Sansi Pseud, Address: Laffour, Faculty: (empty), Email Address: contact@sanipseud.me, Phone Number: 0940253621, Age: 22 years, and Created On: December 19, 2020 09:37:53 AM. At the bottom, there are two buttons: 'Create Another Student' and 'Go to My Record Page'.

Figure 1: Student form validation and details

Lab 19: Dependency Injection in ASP.NET Core

Title

Create a project (WebApp2BySamir) to illustrate dependency injection with different service lifetimes.

Theory

Dependency Injection is a design pattern where objects receive dependencies from external sources. ASP.NET Core has built-in DI container with three service lifetimes: **Transient** (new instance every time), **Scoped** (one instance per request), **Singleton** (single instance for app lifetime).

Service Registration (Program.cs)

```
1 // Transient: New instance every time
2 builder.Services.AddTransient<ITimeService, TimeService>();
3
4 // Scoped: Same instance within a request
5 builder.Services.AddScoped<ILoggerService, LoggerService>();
6
7 // Singleton: Single instance throughout app
8 builder.Services.AddSingleton<IGreetingService, GreetingService>();
```

Constructor Injection

```
1 public class DIController : Controller
2 {
3     private readonly IGreetingService _greetingService;
4     private readonly ITimeService _timeService;
5
6     public DIController(IGreetingService greeting,
7                         ITimeService time)
8     {
9         _greetingService = greeting;
10        _timeService = time;
11    }
12 }
```

Services Implemented

- **IGreetingService** (Singleton): Provides greeting messages
- **IDataService** (Singleton): Manages student list, shows service creation time
- **ILoggerService** (Scoped): Logs activities with instance ID
- **ITimeService** (Transient): Returns current time, demonstrates new instances

Output

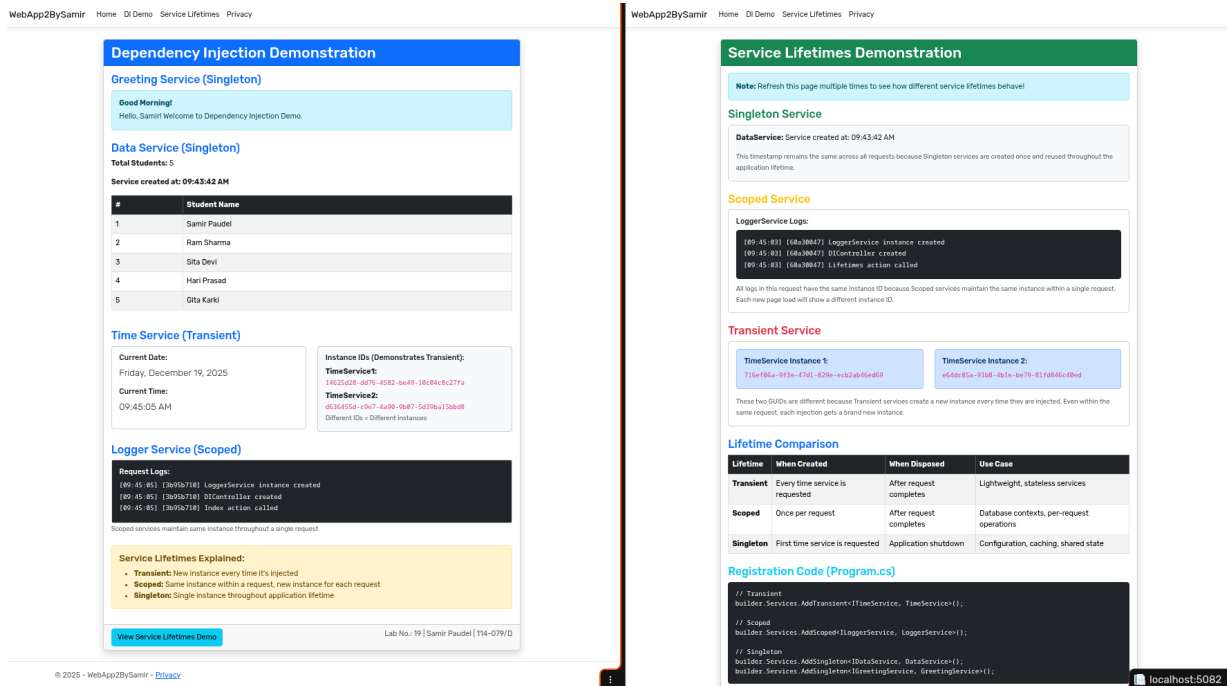


Figure 2: Dependency Injection service lifetimes

Lab 20: Database CRUD Operations with SQL Server

Title

Console application (DatabaseCRUDBySamir) demonstrating INSERT, READ, UPDATE, DELETE operations with SQL Server.

Theory

CRUD operations are fundamental database operations. ADO.NET provides classes like SqlConnection, SqlCommand, and SqlDataReader for database access. Parameterized queries prevent SQL injection. The application uses try-catch for error handling and using statements for resource disposal.

Database Table Structure

```
1 CREATE TABLE Students (
2     StudentID INT PRIMARY KEY IDENTITY(1,1),
3     Name NVARCHAR(100) NOT NULL,
4     Email NVARCHAR(100) NOT NULL,
5     Age INT NOT NULL,
6     Course NVARCHAR(50) NOT NULL,
7     City NVARCHAR(50) NOT NULL,
8     CreatedDate DATETIME DEFAULT GETDATE()
9 );
```

CRUD Operations

1. INSERT:

```
1 string query = @"INSERT INTO Students (Name, Email, Age, Course, City)
2     VALUES (@Name, @Email, @Age, @Course, @City)";
3 cmd.Parameters.AddWithValue("@Name", name);
4 cmd.ExecuteNonQuery();
```

2. READ:

```
1 string query = "SELECT * FROM Students ORDER BY StudentID";
2 using (SqlDataReader reader = cmd.ExecuteReader())
3 {
4     while (reader.Read())
5         Console.WriteLine($"{reader["Name"]} - {reader["Email"]}");
6 }
```

3. UPDATE:

```
1 string query = @"UPDATE Students SET Name = @Name, Email = @Email
2 WHERE StudentID = @StudentID";
```

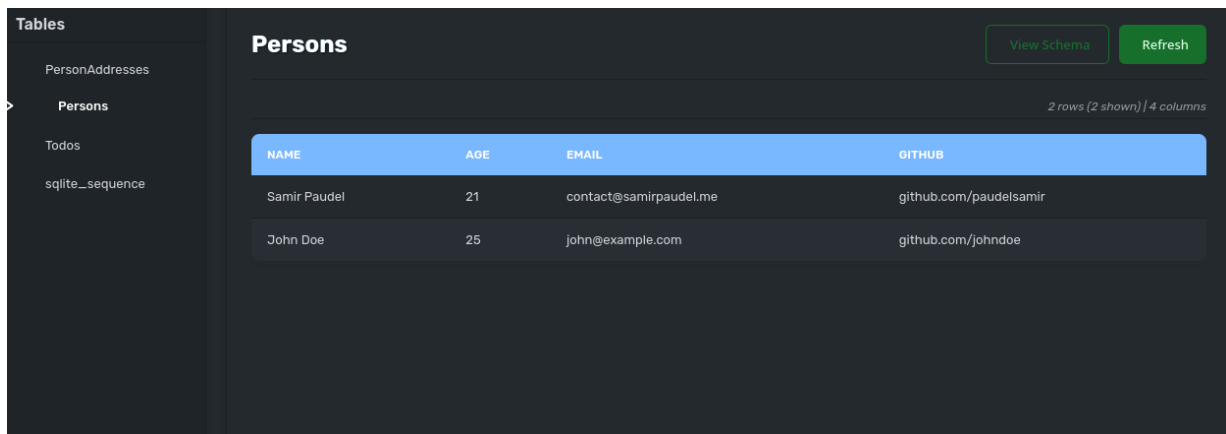
4. DELETE:

```
1 string query = "DELETE FROM Students WHERE StudentID = @StudentID";
```

Features

- Auto-creates database and table on first run
- Menu-driven interface (Insert, Read All, Read by ID, Update, Delete)
- Parameterized queries for security
- Comprehensive error handling
- Display records in formatted table

Output



The screenshot shows a web application interface for a database. On the left is a sidebar with a 'Tables' section containing a list of tables: 'PersonAddresses', 'Persons' (which is selected and highlighted), 'Todos', and 'sqlite_sequence'. The main area displays the 'Persons' table. At the top right of this area are two buttons: 'View Schema' and 'Refresh'. Below the buttons, it says '2 rows (2 shown) / 4 columns'. The table itself has four columns: 'NAME', 'AGE', 'EMAIL', and 'GITHUB'. It contains two rows of data: one for 'Samir Paudel' (age 21, email contact@samirpaudel.me, github github.com/paudelsamir) and one for 'John Doe' (age 25, email john@example.com, github github.com/johndoe).

NAME	AGE	EMAIL	GITHUB
Samir Paudel	21	contact@samirpaudel.me	github.com/paudelsamir
John Doe	25	john@example.com	github.com/johndoe

Figure 3: Database CRUD operations

Lab 21: ASP.NET Core MVC CRUD with ADO.NET

Title

Create CRUD application using ASP.NET Core MVC template with ADO.NET (WebApp3BySamir).

Theory

ADO.NET provides low-level database access in .NET. Uses SqlConnection for database connection, SqlCommand for executing queries, and SqlDataReader for reading data. Parameterized queries prevent SQL injection. This approach gives full control over SQL and database operations.

Database Structure

```
1 CREATE TABLE Products (  
2     ProductID INT PRIMARY KEY IDENTITY(1,1),  
3     ProductName NVARCHAR(100) NOT NULL,  
4     Price DECIMAL(18,2) NOT NULL,  
5     Quantity INT NOT NULL,  
6     Category NVARCHAR(50),  
7     CreatedDate DATETIME DEFAULT GETDATE()  
8 );
```

Key Components

1. Data Access Layer:

```
1 public bool InsertProduct(Product product)  
2 {  
3     using (SqlConnection conn = new SqlConnection(_connectionString))  
4     {  
5         conn.Open();  
6         string query = @"INSERT INTO Products (ProductName, Price, Quantity, Category, CreatedDate)  
7             VALUES (@ProductName, @Price, @Quantity, @Category, @CreatedDate)";  
8         using (SqlCommand cmd = new SqlCommand(query, conn))  
9         {  
10            cmd.Parameters.AddWithValue("@ProductName", product.ProductName);  
11            cmd.Parameters.AddWithValue("@Price", product.Price);  
12            // ... other parameters  
13            cmd.ExecuteNonQuery();  
14        }  
15    }  
16 }
```

2. Controller Actions:

```
1 [HttpPost]  
2 public IActionResult Create(Product product)  
3 {  
4     if (ModelState.IsValid)  
5     {  
6         if (_dataAccess.InsertProduct(product))  
7             return RedirectToAction(nameof(Index));  
8     }  
9     return View(product);  
10 }
```

Features

- Product management with full CRUD
- Parameterized SQL queries for security
- Auto-creates database and table on startup
- Model validation with Data Annotations
- Bootstrap UI with responsive design
- TempData for success/error messages

Output

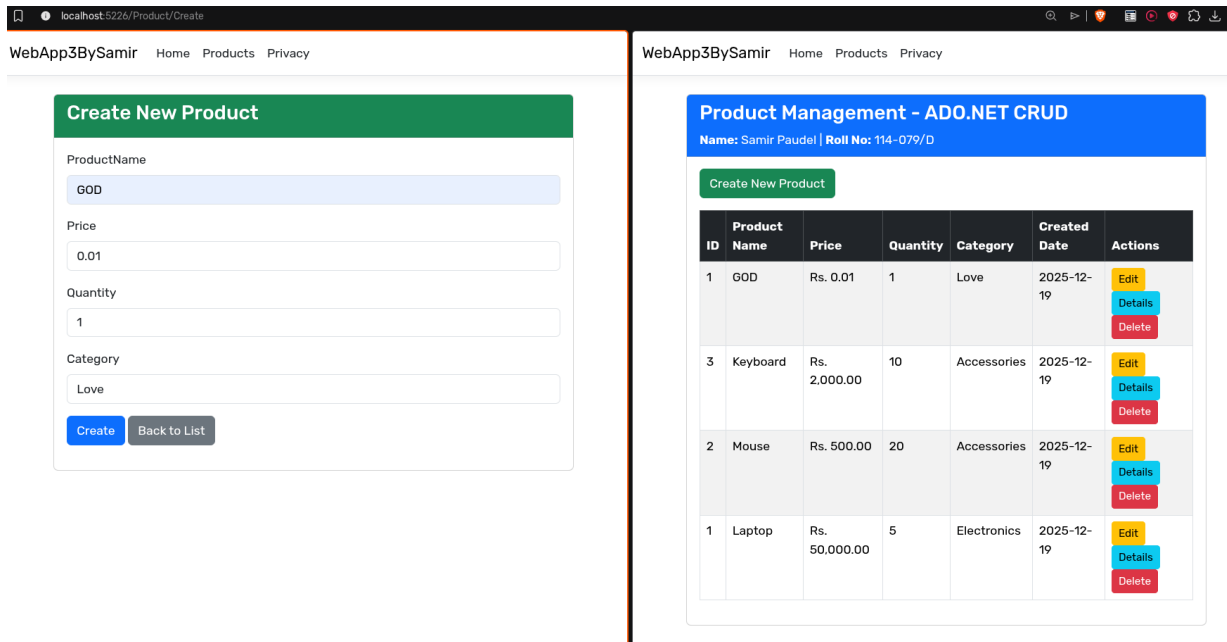


Figure 4: MVC CRUD with ADO.NET

Lab 22: Entity Framework Core - Code First Approach

Title

Create CRUD application using ASP.NET Core MVC with Entity Framework Core Code First (WebApp4BySamir).

Theory

EF Core Code First creates database from C# models. DbContext manages entity objects and database connection. Migrations track schema changes. LINQ queries are translated to SQL automatically. Provides change tracking, lazy loading, and relationship management.

Model Definition

```

1 public class Book
2 {
3     [Key]
4     public int BookId { get; set; }
5
6     [Required, StringLength(200)]
7     public string Title { get; set; }
8
9     [Required, StringLength(100)]
10    public string Author { get; set; }
11
12    [Range(0.01, 999999.99)]
13    public decimal Price { get; set; }
14 }

```

DbContext Configuration

```

1 public class ApplicationDbContext : DbContext
2 {
3     public DbSet<Book> Books { get; set; }
4
5     protected override void OnModelCreating(ModelBuilder modelBuilder)
6     {
7         modelBuilder.Entity<Book>().Property(e => e.Price)
8             .HasColumnType("decimal(18,2)");
9
10        // Seed data
11        modelBuilder.Entity<Book>().HasData(
12            new Book { BookId = 1, Title = "C# Guide", Author = "Samir" }
13        );
14    }
15 }

```

Controller with EF Core

```
1 public async Task<IActionResult> Index()
2 {
3     return View(await _context.Books.OrderBy(b => b.BookId).ToListAsync());
4 }
5
6 [HttpPost]
7 public async Task<IActionResult> Create(Book book)
8 {
9     if (ModelState.IsValid)
10     {
11         _context.Add(book);
12         await _context.SaveChangesAsync();
13         return RedirectToAction(nameof(Index));
14     }
15     return View(book);
16 }
```

Migration Commands

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Features

- Entity Framework Core with async operations
- Auto-migration on application startup
- LINQ queries (OrderBy, Where, FirstOrDefault)
- DbContext dependency injection
- Seed data for initial records
- Book management CRUD operations

Output

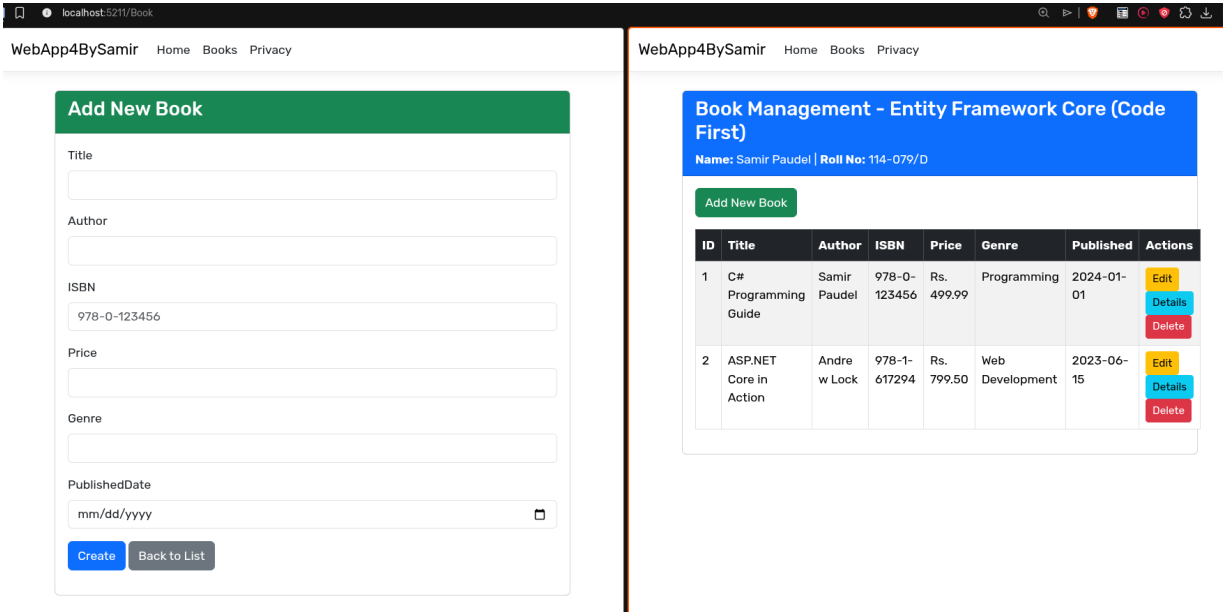


Figure 5: EF Core Code First

Lab 23: Entity Framework Core - Database First Approach

Title

Create ASP.NET Core application demonstrating EF Core Database First (WebApp5BySamir).

Theory

Database First starts with existing database and generates models. Uses scaffolding (reverse engineering) to create entity classes and DbContext from database schema. Useful for legacy databases or when database design is done separately.

Workflow

1. Create database and tables in SQL Server
2. Install EF Core tools and SqlServer provider
3. Run scaffold command to generate models
4. Use generated DbContext in controllers

Database Creation

```
1 CREATE DATABASE WebApp5DB;
2 USE WebApp5DB;
3
4 CREATE TABLE Employees (
5     EmployeeID INT PRIMARY KEY IDENTITY(1,1),
6     FirstName NVARCHAR(50) NOT NULL,
7     LastName NVARCHAR(50) NOT NULL,
8     Email NVARCHAR(100),
9     Department NVARCHAR(50),
10    Salary DECIMAL(18,2),
11    HireDate DATETIME DEFAULT GETDATE()
12 );
```

Scaffold Command

```
1 dotnet ef dbcontext scaffold
2 "Server=localhost;Database=WebApp5DB;Trusted_Connection=True;"
3 Microsoft.EntityFrameworkCore.SqlServer
4 --output-dir Models
5 --context EmployeeDbContext
```

This generates:

- Employee.cs - Model with properties matching table columns
- EmployeeDbContext.cs - DbContext with DbSet and configurations

Generated Model Example

```
1 public partial class Employee
2 {
3     public int EmployeeId { get; set; }
4     public string FirstName { get; set; }
5     public string LastName { get; set; }
6     public string? Email { get; set; }
7     public decimal? Salary { get; set; }
8     public DateTime? HireDate { get; set; }
9 }
```

Usage in Controller

```
1 public class EmployeeController : Controller
2 {
3     private readonly EmployeeDbContext _context;
4
5     public async Task<IActionResult> Index()
6     => View(await _context.Employees.ToListAsync());
7 }
```

Lab 24: ASP.NET Core Web API with Swagger

Title

Create Web API using ASP.NET Core and Entity Framework Core with Postman and Swagger testing (WebApiBySamir).

Theory

Web APIs provide HTTP endpoints for client applications. RESTful design uses HTTP verbs (GET, POST, PUT, DELETE). Swagger/OpenAPI generates interactive API documentation. Returns JSON/XML data instead of HTML views.

API Controller

```
1 [Route("api/[controller]")]
2 [ApiController]
3 public class TasksController : ControllerBase
4 {
5     private readonly TaskDbContext _context;
6
7     // GET: api/Tasks
8     [HttpGet]
9     public async Task<ActionResult<IEnumerable<Task>>> GetTasks()
10     => await _context.Tasks.ToListAsync();
11
12     // POST: api/Tasks
13     [HttpPost]
14     public async Task<ActionResult<Task>> PostTask(Task task)
15     {
16         _context.Tasks.Add(task);
17         await _context.SaveChangesAsync();
18         return CreatedAtAction(nameof(GetTask), new { id = task.TaskId }, task);
19     }
20
21     // PUT: api/Tasks/5
22     [HttpPut("{id}")]
23     public async Task<ActionResult> PutTask(int id, Task task)
24     {
25         _context.Entry(task).State = EntityState.Modified;
26         await _context.SaveChangesAsync();
27         return NoContent();
28     }
29 }
```

Swagger Configuration (Program.cs)

```
1 builder.Services.AddSwaggerGen(c =>
2 {
3     c.SwaggerDoc("v1", new OpenApiInfo
4     {
5         Title = "Task API by Samir Paudel",
6         Version = "v1"
7     });
8 });
9
10 app.UseSwagger();
11 app.UseSwaggerUI(c =>
12 {
13     c.SwaggerEndpoint("/swagger/v1/swagger.json", "Task API v1");
14     c.RoutePrefix = string.Empty; // Swagger UI at root
15 });
```

Testing Methods

Swagger UI:

- Navigate to <https://localhost:5001>
- Click "Try it out" on any endpoint
- Enter parameters, click "Execute"
- View response body and status code

Postman Testing:

```
1 GET https://localhost:5001/api/tasks
2
3 POST https://localhost:5001/api/tasks
4 Body (JSON):
5 {
```

```

6  "title": "Complete Lab 24",
7  "priority": "High",
8  "isCompleted": false
9  }
10
11 PUT https://localhost:5001/api/tasks/1
12 DELETE https://localhost:5001/api/tasks/1

```

HTTP Status Codes Used

- 200 OK - Successful GET
- 201 Created - Successful POST
- 204 No Content - Successful PUT/DELETE
- 400 Bad Request - Validation errors
- 404 Not Found - Resource doesn't exist

Output

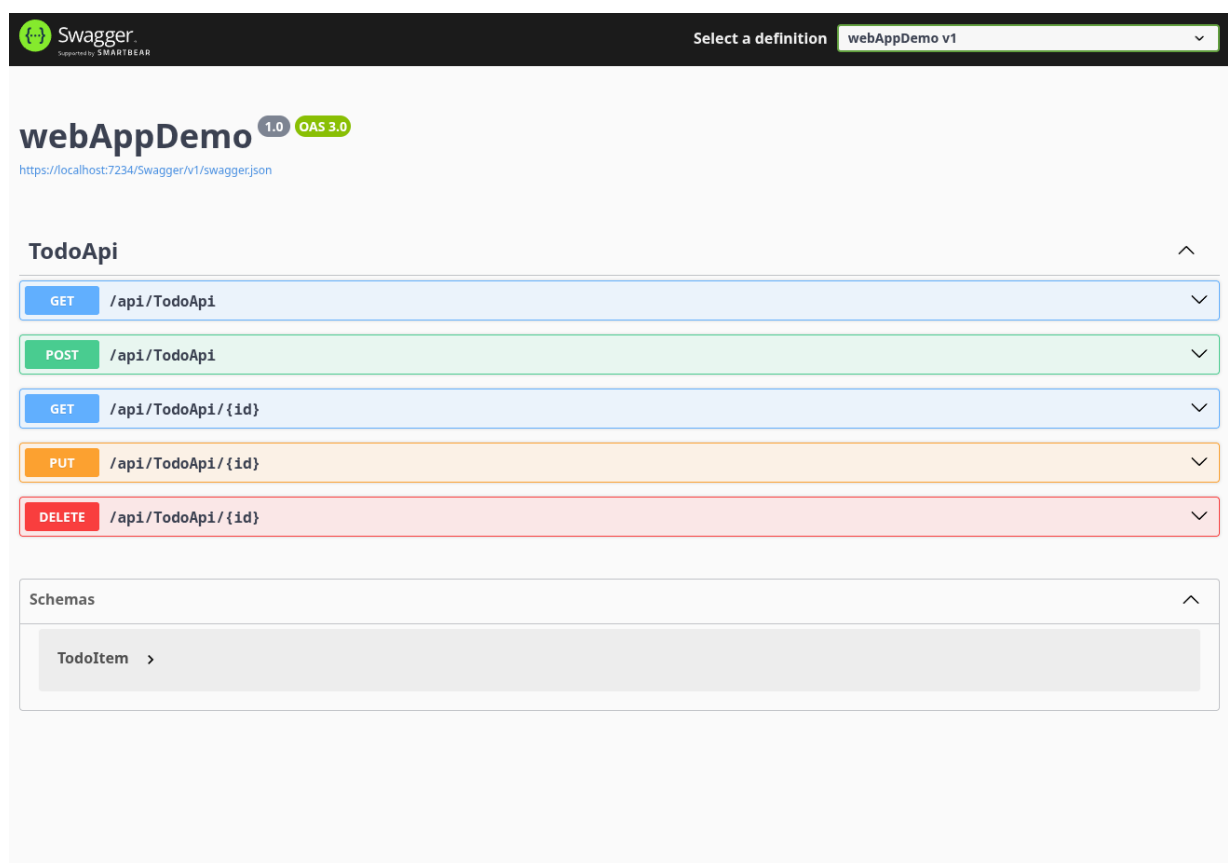


Figure 6: Web API with Swagger

Lab 25: State Management in ASP.NET Core

Title

Demonstrate state management using Session, HttpContext, TempData, Cookies, Query Strings, and Hidden Fields (WebApp6BySamir).

Theory

State management preserves data across HTTP requests. Server-side (Session, TempData) stores data on server. Client-side (Cookies, Query Strings, Hidden Fields) stores on client. Each has different scope, lifetime, and use cases.

Implementation

1. Session State:

```
1 // Configure in Program.cs
2 builder.Services.AddDistributedMemoryCache();
3 builder.Services.AddSession(options => {
4     options.IdleTimeout = TimeSpan.FromMinutes(30);
5 });
6 app.UseSession();
7
8 // Usage
9 HttpContext.Session.SetString("Username", "Samir Paudel");
10 var username = HttpContext.Session.GetString("Username");
```

2. HttpContext.Items:

```
1 HttpContext.Items["RequestTime"] = DateTime.Now;
2 var time = HttpContext.Items["RequestTime"];
```

3. TempData:

```
1 TempData["SuccessMessage"] = "Data saved!";
2 var msg = TempData["SuccessMessage"]; // Auto-deleted after read
3 TempData.Keep("SuccessMessage"); // Keep for next request
```

4. Cookies:

```
1 Response.Cookies.Append("Theme", "Dark", new CookieOptions
2 {
3     Expires = DateTimeOffset.Now.AddDays(7),
4     HttpOnly = true
5 });
6 Request.Cookies.TryGetValue("Theme", out string theme);
```

5. Query Strings:

```
1 // In view: <a href="/Product/Details?id=58&name=Laptop">
2 public IActionResult Details(int id, string name) { }
```

6. Hidden Fields:

```
1 <input type="hidden" name="ProductId" value="@Model.ProductId" />
```

Comparison

Technique	Storage	Lifetime	Use Case
Session	Server	Until timeout	User auth, cart
HttpContext.Items	Server	Single request	Middleware data
TempData	Server	One redirect	Flash messages
Cookies	Client	Set expiration	Preferences
Query String	Client	Single request	Filtering
Hidden Field	Client	Form post	CSRF tokens

Output

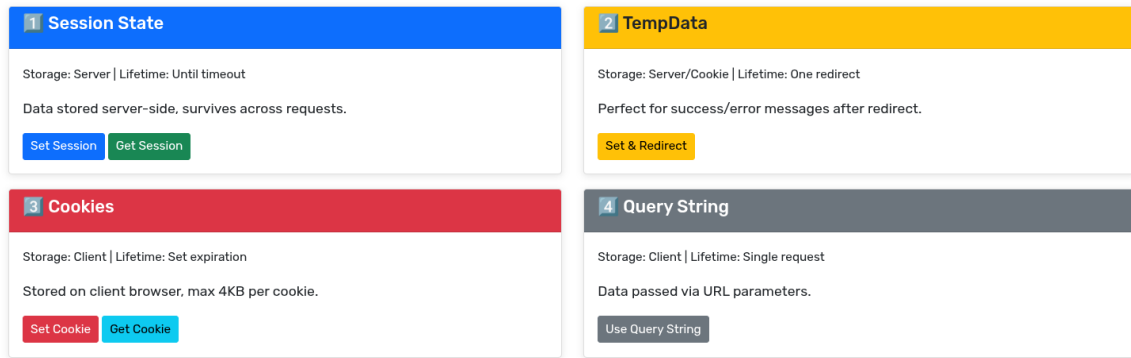


Figure 7: State Management techniques

Lab 26: Client-Side Development

Part A: jQuery Form Validation

Overview: Signup form with client-side validation using jQuery. Display name and roll at top.

Key Features:

- Real-time validation on blur event
- Regex patterns for email and phone
- Password confirmation matching
- Error messages below fields (red color)
- Success message on valid submission

Validation Example:

```

1 function validateEmail() {
2     let email = $('#email').val().trim();
3     let regex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$/;
4     if (!regex.test(email)) {
5         $('#email-error').show();
6         return false;
7     }
8     $('#email-error').hide();
9     return true;
10 }
11
12 $('#signupForm').submit(function(e) {
13     e.preventDefault();
14     if (validateEmail() && validatePassword()) {
15         $('#success-message').text('Signup successful!');
16     }
17 });

```

Part B: Angular Calculator App

Project: AngularAppBySamir

Components:

- Navbar with Home and Calculator menus
- Footer with copyright and current year
- Home page with profile photo
- Calculator with two inputs, dropdown, compute button

Calculator Component:

```

1 export class CalculatorComponent {
2     number1: number = 0;
3     number2: number = 0;
4     operation: string = 'add';
5     result: number | null = null;

```

```

6   compute(): void {
7       switch(this.operation) {
8           case 'add': this.result = this.number1 + this.number2; break;
9           case 'subtract': this.result = this.number1 - this.number2; break;
10          case 'multiply': this.result = this.number1 * this.number2; break;
11      }
12  }
13  }
14  }

```

Two-way Binding:

```

1  <input type="number" [(ngModel)]="number1">
2  <select [(ngModel)]="operation">
3      <option value="add">Add</option>
4      <option value="subtract">Subtract</option>
5      <option value="multiply">Multiply</option>
6  </select>
7  <button (click)="compute()">Compute</button>
8  <h3 *ngIf="result !== null">Result: {{ result }}</h3>

```

Part C: React Calculator App

Project: ReactAppBySamir

React Hooks Implementation:

```

1  function Calculator() {
2      const [number1, setNumber1] = useState(0);
3      const [number2, setNumber2] = useState(0);
4      const [operation, setOperation] = useState('add');
5      const [result, setResult] = useState(null);
6
7      const compute = () => {
8          let calcResult;
9          switch(operation) {
10             case 'add': calcResult = number1 + number2; break;
11             case 'subtract': calcResult = number1 - number2; break;
12             case 'multiply': calcResult = number1 * number2; break;
13          }
14          setResult(calcResult);
15      };
16
17      return (
18          <div>
19              <input value={number1} onChange={(e) => setNumber1(e.target.value)} />
20              <button onClick={compute}>Compute</button>
21              {result !== null && <h3>Result: {result}</h3>}
22          </div>
23      );
24  }

```

Output

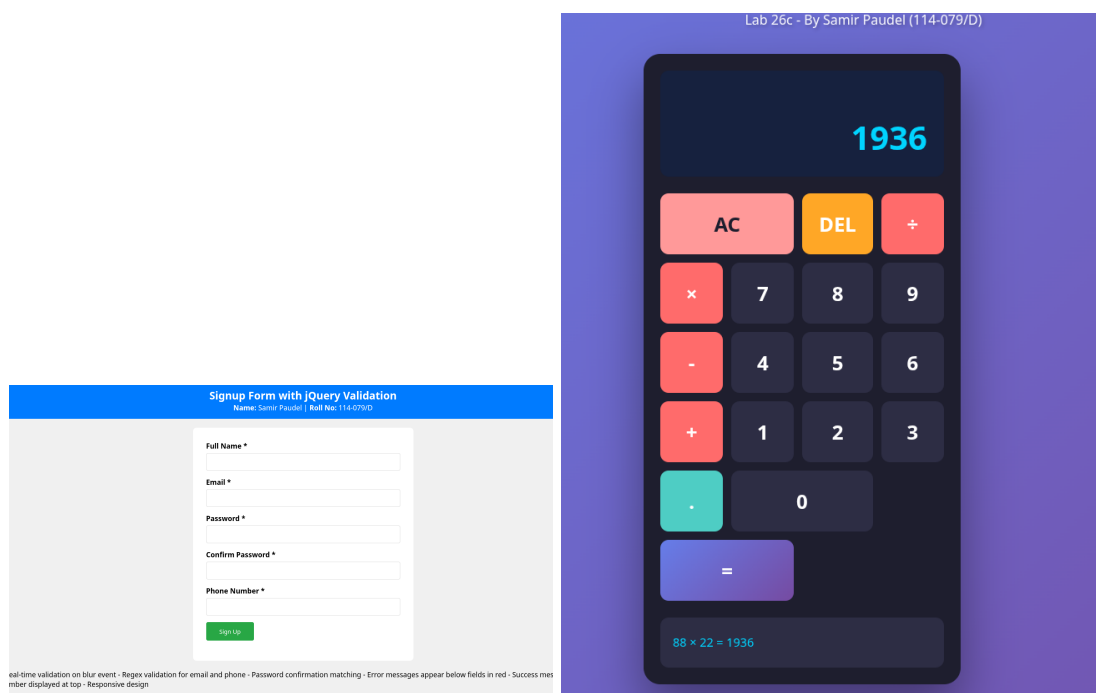


Figure 8: jQuery validation and React calculator

Lab 27: Authentication & Authorization

Title

Create ASP.NET Core application demonstrating authentication and authorization (WebApp7BySamir).

Theory

Authentication: Verifies WHO the user is (Login/Register). **Authorization:** Determines WHAT the user can access (Roles/Policies). ASP.NET Core Identity provides user management, password hashing, role management, and claims-based security.

Configuration

```
1 // Program.cs
2 builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
3 {
4     options.Password.RequiredLength = 6;
5     options.Password.RequireDigit = true;
6 })
7 .AddEntityFrameworkStores<ApplicationDbContext>();
8
9 builder.Services.AddAuthorization(options =>
10 {
11     options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
12 });
```

User Registration

```
1 [HttpPost]
2 public async Task<IActionResult> Register(RegisterViewModel model)
3 {
4     var user = new ApplicationUser
5     {
6         UserName = model.Email,
7         Email = model.Email,
8         FullName = model.FullName
9     };
10
11     var result = await _userManager.CreateAsync(user, model.Password);
12
13     if (result.Succeeded)
14     {
15         await _userManager.AddToRoleAsync(user, "Student");
16         await _signInManager.SignInAsync(user, isPersistent: false);
17         return RedirectToAction("Index", "Home");
18     }
19     return View(model);
20 }
```

Login

```
1 var result = await _signInManager.PasswordSignInAsync(
2     model.Email, model.Password, model.RememberMe, lockoutOnFailure: false);
```

Authorization Attributes

```
1 [Authorize] // Requires login
2 public class DashboardController : Controller { }
3
4 [Authorize(Roles = "Admin")] // Requires Admin role
5 public class AdminController : Controller { }
6
7 [Authorize(Policy = "AdminOnly")] // Requires policy
8 public IActionResult ManageUsers() { }
```

Check in Views

```
1 @if (User.Identity.IsAuthenticated)
2 {
3     <p>Welcome, @User.Identity.Name!</p>
4 }
5
6 @if (User.IsInRole("Admin"))
7 {
8     <a asp-controller="Admin">Admin Panel</a>
9 }
```

Seed Roles

```
1 string[] roles = { "Admin", "Teacher", "Student" };
2 foreach (var role in roles)
3 {
4     if (!await _roleManager.RoleExistsAsync(role))
5         await _roleManager.CreateAsync(new IdentityRole(role));
6 }
```

Features

- User registration with password hashing
- Login with cookie authentication
- Role-based authorization (Admin, Teacher, Student)
- Policy-based authorization
- Logout functionality
- Access denied page
- Identity database tables auto-created

Output

You are not logged in. Please register or login to continue.

New User?

Create a new account

Register

Have Account?

Sign in to your account

Login

Demo Credentials

Admin	admin@lab27.com	Admin@123
Teacher	teacher@lab27.com	Teacher@123
Student	student@lab27.com	Student@123

How It Works

- **Authentication:** Verify who you are (Login/Register)
- **Authorization:** Determine what you can access (Roles)
- **Roles:** Admin, Teacher, Student - different access levels
- **Cookie-based:** Session stored securely with expiration
- **Password Hashing:** Passwords hashed with ASP.NET Identity

Figure 9: Authentication & Authorization

Lab 28: Hosting ASP.NET Core Application

Title

Host Lab 23 application (WebApp5BySamir) on free hosting platform with complete documentation.

Hosting Option

Azure App Service

1. Create Azure account (free tier)
2. Publish app: `dotnet publish -c Release`
3. Create Web App in Azure Portal
4. Create Azure SQL Database
5. Deploy via Azure CLI or Visual Studio
6. Configure connection string in App Settings

Deployment Commands (Azure)

```
1 # Create resource group
2 az group create --name WebApp5RG --location eastus
3
4 # Create App Service plan
5 az appservice plan create --name WebApp5Plan --resource-group WebApp5RG --sku F1
6
7 # Create web app
8 az webapp create --name webapp5bysamir --resource-group WebApp5RG --plan WebApp5Plan
9
10 # Deploy
11 az webapp deploy --resource-group WebApp5RG --name webapp5bysamir --src-path ./publish.zip
```

Pre-Deployment Checklist

- Set environment to Production
- Configure production database connection
- Disable detailed errors (use custom error pages)
- Enable HTTPS redirection
- Test in Release mode locally
- Apply all database migrations
- Use environment variables for secrets

Post-Deployment Verification

- Test all CRUD operations
- Verify database connection
- Check application logs
- Test on mobile/different browsers
- Monitor performance

Platform Comparison

Platform	Pros	Database
Azure	Professional, scalable	SQL Azure
Somee	Easy setup	MSSQL included
Heroku	Simple deployment	Add-ons
Railway	GitHub integration	Multiple options

Output

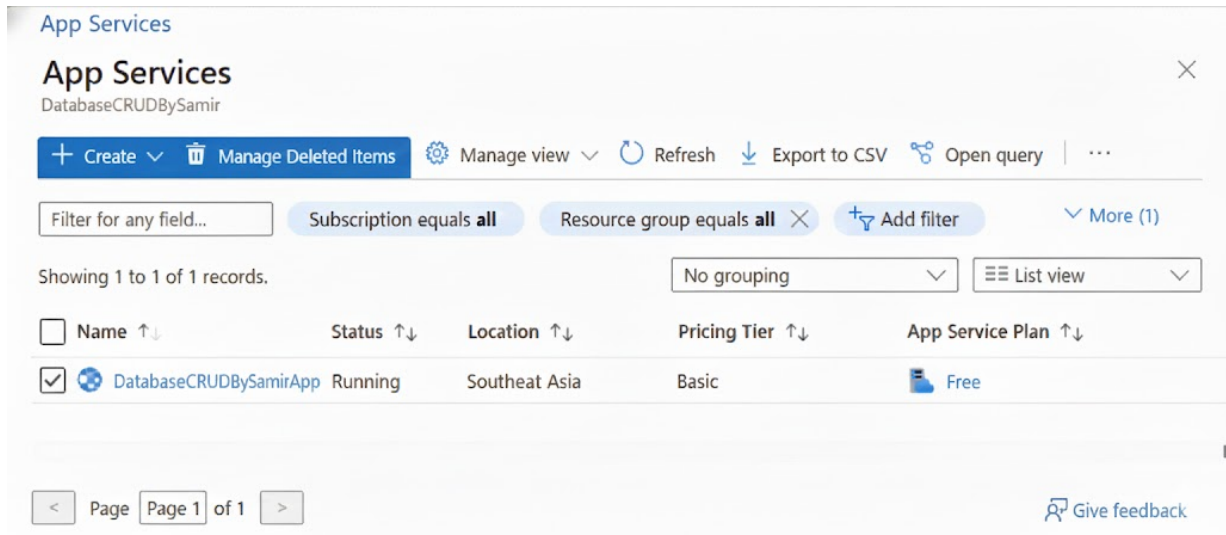


Figure 10: Hosted application on Azure